

Req2CAD: bridging functional requirements and parametric CAD models to support conceptual 3D design

Qianzhi Jing
College of Computer Science and
Technology
Zhejiang University
Hangzhou, China
jingqz@zju.edu.cn

Hankai Lu
Zhejiang University
Hangzhou, China
3220103695@zju.edu.cn

Shuojin Huang
School of Airspace Science and
Engineering
Shandong University
Weihai, China
shuojin.huang@mail.sdu.edu.cn

Peter Childs
Dyson School of Design Engineering
Imperial College London
London, United Kingdom
p.childs@imperial.ac.uk

Liuqing Chen*
College of Artificial Intelligence
Zhejiang University
Hangzhou, China
chenlq@zju.edu.cn

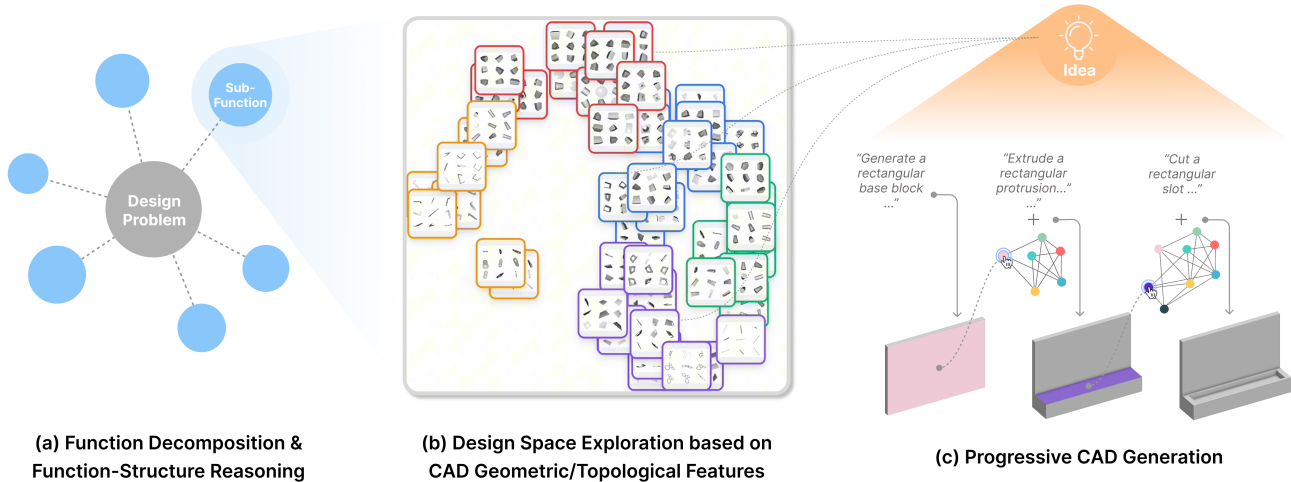


Figure 1: Three main features of Req2CAD that support conceptual CAD. (a) Performing function decomposition from a design problem and reasoning component-level CAD structures for each sub-function. (b) Arranging and clustering inferred CAD components based on geometric and topological similarity to support design space exploration and inspire ideas. (c) Progressively generating CAD models using text prompts and user-selected working surfaces through graphical interaction.

Abstract

Conceptual CAD requires transforming functional requirements into parametric 3D models, yet existing systems have steep learning curves and limit creativity through premature fixation. Generative AI shows promise in producing diverse alternatives, while current methods mainly reconstruct CAD modeling sequences of existing designs, making them unsuitable for early stages where ideas are

vague and intent is difficult to express. We present Req2CAD, an interactive system that enables designers to progress from design problems toward conceptual CAD models through functional decomposition, function-structure reasoning, and component-level CAD creation and iteration. Req2CAD introduces a data annotation pipeline that maps functional requirements to the 3D structural design space, a dual-feature CAD representation to support design space exploration and CAD ideation, and a progressive CAD generation method that enables rapid CAD model creation through multi-modal intent expression. A technical evaluation and user study demonstrate the effectiveness of Req2CAD, highlighting its potential for human-AI co-creation.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. CHI '26, Barcelona, Spain

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2278-3/26/04
<https://doi.org/10.1145/3772318.3791949>

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; • **Computing methodologies** → **Parametric curve and surface models**.

Keywords

Conceptual 3D Design, CAD Generation and Iteration, Creativity Support Tool

ACM Reference Format:

Qianzhi Jing, Hankai Lu, Shuojin Huang, Peter Childs, and Liuqing Chen. 2026. Req2CAD: bridging functional requirements and parametric CAD models to support conceptual 3D design. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 25 pages. <https://doi.org/10.1145/3772318.3791949>

1 Introduction

Conceptual product design begins with a design problem and requires designers to transform high-level functional requirements into candidate solutions [24, 81]. Although early-stage ideation is often supported by sketching and other rapid 2D techniques, many products, such as phone holders, lamp frames and mechanical tools, naturally require 3D representations to convey the concepts and rely on conceptual Computer-Aided Design (CAD), in which designers create coarse, exploratory 3D models that allow them to reason about 3D forms and feasibility, spatial and interactive relationships between multiple components, or functional fit [37, 49, 64]. CAD systems, such as Solidworks [76], Onshape [61] and FreeCAD [20], serve as foundational tools in product design, enabling designers to construct parametric 3D components through a combination of 2D sketches (e.g., circles, lines, splines) and 3D operations (e.g., extrusion, loft, fillet), and subsequently assemble these components into a complete parametric 3D object. However, creating conceptual 3D models within CAD systems is highly challenging, as designers must not only ensure that the proposed design solution meets the functional requirements of the design problem, but also consider how this solution can be realized through parametric modeling operations. The increasing complexity of CAD interfaces places high demands on designers' proficiency and expertise, thereby limiting the accessibility of these systems for novice designers or non-expert users and reducing overall efficiency. This complexity also shifts attention away from the original design goals: instead of exploring broader design spaces, designers may become trapped in local parameter adjustments, leading to premature fixation [44, 64, 67]. As a result, the functionality and novelty of parametric 3D designs are often constrained.

One direction to enhance designers' exploration of broader design spaces is to employ generative design methods (e.g., topology optimization) [1, 34, 78] or deep generative model [42, 95] to generate diverse structural variations for inspiration. Some studies [18, 53] have further developed interactive systems that help designers control design parameters to visualize large-scale generated designs. However, these systems typically rely on expert-defined design variables and optimization goals for specific categories (e.g., chairs, perfume bottles) [1, 53], or on AI models trained on large-scale 3D datasets within such domains [18, 42]. As a result, their

applicability to general design problems is limited, as designers often struggle to directly translate complex design problems into specific design variables or optimization objectives [26], and the explored design space remains constrained by baseline-dependent parameterizations.

Recently, generative AI (GenAI) such as Stable Diffusion [66] and DALL-E 3 [56] has been introduced into 3D design workflows to inspire novel directions [49, 96]. Yet, their outputs are typically in the form of images or 3D meshes, which allow little local editing and cannot be directly integrated into iterative CAD workflows, leaving designers to manually construct CAD models. To better support efficient CAD creation, researchers have reformulated CAD generation as a sequence prediction task over 2D sketch and 3D extrude operations [36, 86, 88, 100], and more recently leveraged Large Language Models (LLMs) to produce CAD code from text prompts [2, 5, 52]. While effective, these AI-assisted methods largely focus on the performance of reconstructing the parameters and construction processes of existing CAD models from 2D images or 3D representations. Even text-based CAD generation methods typically require prompts containing detailed and precise design information [2, 36]. Yet, in the early stages of 3D design, concepts are often vague and must be refined through iterative exploration. The inherent spatial and structural complexity of 3D objects further exacerbates the challenge, making it difficult for novice designers to accurately express initial ideas or subsequent design intentions through text alone [21]. This gap highlights a key challenge: how to enable designers, particularly novice designers, to effectively reason from a design problem to conceptual CAD that meet specific requirements with the assistance of GenAI.

To address this challenge, we introduce **Req2CAD**, a GenAI-based interactive system that supports novice designers in creating CAD models during the early conceptual stage. With Req2CAD, designers can start from a given design problem, decompose the overall product function into basic sub-functions with the assistance of an LLM, reason from these basic functions to component-level structural design concept, and then effectively create and iterate parametric CAD components based on the design concept. These components can finally be exported to CAD software such as Onshape for assembly, enabling the construction of complete parametric 3D products. Specifically, we integrate three technical and interaction innovations into our system to enhance the above design workflow. First, we introduce a data annotation pipeline that leverages vision-language models (VLMs) and LLMs to generate functional descriptions for 128k CAD components in Text2CAD dataset [36], thereby establishing a mapping from basic functions in the problem space to broad component-level structural designs in the solution space, assisting designers in function-to-structure reasoning. Second, we develop a dual-feature CAD representation that combines boundary representation (B-rep) graphs (for topological features) with point cloud embeddings (for geometric features), enabling designers to efficiently navigate and compare within large design spaces, as well as identify and combine typical structural patterns. Third, we design an LLM-based progressive CAD generation method that integrates text prompts with graphical interactions, allowing designers to accurately express design intent through multi-modal input and rapidly generate and iterate component-level CAD models.

We conducted a technical evaluation to validate Req2CAD’s technical features, followed by a within-subject user study with twelve novice designers. Compared to the baseline system, expert assessments of the generated 3D designs indicate that Req2CAD enhances functionality, novelty, and usefulness. Participant feedback further highlights the system’s usability and its distinct advantages in supporting conceptual 3D design tasks.

The contributions of this work are summarized as follows:

- We introduce Req2CAD, a novel interactive system that assists novice designers to create and iterate CAD model for specific design problems during early conceptual design.
- We develop a data annotation pipeline to produce a function-structure knowledge base that links functional descriptions to 128k+ CAD components, and propose a dual-feature CAD representation capturing both geometric and topological information for enhancing design space exploration and typical structure pattern identification, thereby supporting designers in ideating design concepts.
- We present a novel LLM-based CAD generation method that integrates multi-modal inputs to enhance designers’ intent expression and employs progressive CAD code generation to enable the rapid CAD model creation from textual design concepts.
- We conduct comprehensive technical and user evaluations, validating Req2CAD’s usability and demonstrating its advantages in conceptual CAD and human-AI co-creation.

2 Related Work

2.1 3D Conceptual Design

Conceptual design constitutes a pivotal early stage in the product development process, where the primary task is to translate abstract design requirements into candidate solution spaces [24, 72]. As outlined in the classical framework of Pahl and Beitz [57], this stage typically encompasses functional analysis, the generation and evaluation of solution concepts, and the gradual refinement of ideas into concrete designs. The process requires designers to balance functional realization, structural feasibility, and innovation. In practice, designers employ several complementary modes of expression during concept ideation and refinement, including verbal or textual descriptions, hand-drawn sketches, and the construction of conceptual 3D models using CAD tools [64]. While traditional 2D sketches are effective for quickly externalizing ideas, they are limited in expressing spatial depth and geometric constraints. Building on these 2D representations, conceptual CAD allows designers to apply constraints and dimensions and bring their design into 3D through operations like extrusion, lofting, and revolving. This 3D conceptual design offers a more immediate representation of a product’s spatial characteristics and functional mechanisms [64], enable designers to assess form and proportion while also reasoning about assembly relationships among components [87, 98]. At the same time, 3D conceptual design entails greater complexity, such as modeling geometric constraints, reasoning about topological structures, and coordinating design decisions across multiple scales [65]. Addressing these challenges—by lowering the barriers of 3D modeling and improving the efficiency of creative expression

and exploration in the conceptual stage—has remained a central concern in creativity support research.

Early research on supporting 3D design primarily focused on generative design based on topology optimization [7, 16, 54, 80], genetic algorithms [79] and heuristic algorithms [43]. In these approaches, designers define objective functions, constraints, and performance metrics, while the system automatically generates and evaluates a large number of candidate designs. DreamSketch [34] and Alcaide-Marzal et al. [1] demonstrated that generative design has the potential to accelerate iteration and expand the design space in domains such as structural and aesthetic form optimization for product and mechanical design. GEM-NI [94], DreamLens [53], and Juxtaform [58] further improved candidate search, filtering, and visualization, enabling designers to more effectively explore solution spaces. These techniques are gradually being integrated into commercial 3D CAD systems such as SolidWorks [22] and Autodesk Fusion 360 [4]. However, they typically rely on complex parametric modeling and domain expertise. For novice designers, it remains difficult to translate abstract functional requirements or design intent into well-defined objective functions. Moreover, the black-box nature of these methods often prevents designers from forming accurate mental models of how the system operates, thereby limiting their creative use [26].

With the success of GenAI in producing realistic 2D images and 3D meshes from natural language descriptions, recent research has shifted toward incorporating GenAI into conceptual design workflows. These works aim to assist designers in generating design concept in forms of textual description [13, 15, 71], 2D appearance image [14, 38], and 3D shape mesh [18, 96] based on a given design problem. For instance, 3DALL-E [49] leverages LLMs to generate design keywords, followed by text-to-image models to produce reference images for CAD modeling. FusionProtector [96] combines image-to-3D models with XR technologies, enabling rapid physical-to-virtual 3D prototypes transition. GenPara [18] adopts a part-level text-to-3D model [42] to generate chair meshes and uses a fine-tuned LLM to generate skeletal parameters of mesh part, supporting localized text-driven refinements of 3D designs. Despite these advances, such methods typically output images or 3D meshes that lack editability and reusability, serving only as indirect design references, while the creation of parameterized CAD models still requires manual work by designers. Moreover, these approaches mainly focus on appearance modeling, overlooking whether the generated designs meet functional requirements or maintain structural soundness. As a result, they may violate geometric and physical consistency or manufacturability constraints, limiting their applicability in downstream engineering processes such as fabrication, assembly, and simulation.

2.2 Datasets and Generative Models for CAD

The automatic generation of high-quality CAD models has emerged as a critical challenge, demanding both extensive domain expertise and advanced software proficiency from designers. A central research direction involves training generative models with deep learning [88] or reinforcement learning [86], which typically require large-scale datasets for supervision. Some datasets [69, 70] capture 2D sketches in the form of images and drawing operations,

Table 1: Comprehensive comparison of representative CAD generation methods

Method	Input	Output	CAD-based Input	User-intervenable	Generation Mode
SketchGen [59]	–	CAD Sketch Sequence	✓	×	One-step Generation ¹
DAVINCI [33], Vitruvion [70]	CAD Sketch Image	CAD Sketch Sequence	✓	×	One-step Reconstruction ¹
CAD-SIGNet [35]	Point Cloud & User Interaction	CAD Construction Sequence	✓	✓	Progressive Reconstruction ²
DeepCAD [88]	Point Cloud	CAD Construction Sequence	✓	×	One-step Reconstruction
Point2CAD [50]	Point Cloud	B-rep	✓	×	Multi-step Reconstruction ³
ComplexGen [27]	Point Cloud	B-rep chain	✓	×	One-step Reconstruction
HNC [89]	Partial CAD	CAD Construction Sequence	✓	×	One-step Generation
CADDreamer [47]	CAD Image	B-rep	✓	×	Multi-step Reconstruction
CADCrafter [82]	CAD Image	CAD Construction Sequence	✓	×	One-step Reconstruction
CADParser [103]	B-rep	CAD Construction Sequence	✓	×	One-step Reconstruction
ShapeCoder [31]	Unstructured Primitives	CAD Construction Program	✓	×	Multi-step Reconstruction
Text2CAD [36], CADTranslator [46]	Text Prompt	CAD Construction Sequence	×	×	One-step Generation
Query2CAD [5]	Text Prompt & CAD Image	Python Macro Code	✓	×	Iterative Generation
CADCodeVerify [2]	Text Prompt & CAD Image	CADQuery Code	✓	×	Iterative Generation
CADrille [41]	Text Prompt & CAD Image & Point Cloud	CADQuery Code	✓	×	One-step Reconstruction
Ours	Text Prompt & User Interaction	CADQuery Code	×	✓	Progressive Generation ²

Note:

–: Unconditional Generation.

¹ One-step Generation / Reconstruction: Leverage an end-to-end framework or method to perform one-shot CAD model generation or reconstruction.² Progressive Generation / Reconstruction: Incremental generation or reconstruction of a CAD model based on its structural decomposition, with users able to intervene in the process at each step.³ Multi-step Reconstruction: A methodologically multi-stage process executed automatically in sequence, where intermediate steps are not exposed to or influenced by the user, who only receives the final result.

while ABC [39], Thingi10K [102], and CC3D [23] offer extensive collections of 3D CAD models represented as triangle meshes or boundary representations (B-Rep) with parametric curves and surfaces, serving as ground truth for geometric feature detection and shape reconstruction. Subsequently, DeepCAD [88] and Fusion 360 Gallery [86] provide the CAD construction sequences based on 2D sketches and 3D extrusion operations, enabling the modeling and inference of design histories. Building on DeepCAD, Text2CAD [36] further advanced conversational AI for CAD modeling by leveraging VLMs and LLMs to generate multi-level text prompts that describe construction workflows of CAD models with varying complexity. Despite these contributions, existing datasets predominantly emphasize the representation of design solutions while overlooking their origins—the associated design requirements. In

conceptual design, however, designers often invest substantial effort in reasoning and exploration to bridge requirements to feasible solutions. The lack of requirement-level data thus constrains the applicability of current datasets in design knowledge-based CAD generation systems.

Large-scale CAD datasets offer abundant structural references, yet manually identifying typical patterns from large collections is highly inefficient. To facilitate efficient design space exploration, prior work measures similarities between CAD cases using expert-defined attribute parameters [40, 53] or coarse primitives fitted from 3D shapes [18]. The former depends heavily on domain expertise, while the latter relies on pretrained primitive-fitting models [31, 42, 45, 60, 92]. Neither approach captures the topological structure or fine-grained geometry of CAD models. B-Reps encode trimmed parametric surfaces together with their topological adjacencies, and

are widely used for defining [90] or supervising [86] CAD topology in generation tasks. Point clouds are often used to extract geometric features for 3D shape segmentation [62], primitive fitting [45, 60, 92], and reconstruction [27, 41, 50]. While both representations have been shown to effectively model CAD topology and geometry respectively, how to extract computable features from these two representations and perform similarity analysis and clustering for design space exploration remains under-explored. This work takes a step toward addressing this gap.

With the rapid development of generation models and the availability of large-scale CAD datasets, many learning-based CAD generation methods have emerged. Table 1 summarizes the representative methods and compares their key characteristics. Some methods typically take diverse forms of CAD construction information as input, produce either 3D meshes or CAD construction sequences as output, and adopt end-to-end supervised learning to achieve CAD reconstruction during training. For example, SketchGen [59], DAVINCI [33], and Vitruvion [70] generate parameterized and constraint-based sketches from input images. CAD-SIGNet [35], DeepCAD [88], Point2CAD [50], and ComplexGen [27] reconstruct CAD construction sequences or B-Rep chains from point clouds. HNC [89] encodes high-level concepts into a hierarchical neural tree for controllable generation. CADDreamer [47] and CADCrafter [12] generate CAD models from single- or multi-view images, while CADParser [103] and ShapeCoder [31] take B-reps or raw primitive parameters, to produce CAD construction sequences. Since both inputs and outputs in these methods can be serialized, most adopt cross-attention [83] or contrastive learning [63] for cross-modal alignment, while employing Transformers [83] as the backbone for decoding construction sequences. However, these methods primarily aim to reconstruct the modeling process of existing CAD models. They assume that the target model is already available and that designers can supply detailed descriptions of its final geometry—such as point clouds, images, or structured parameters—as input. This assumption conflicts with conceptual design needs, where ideas are vague and exploratory and models must be created from scratch, thereby limiting the real-world applicability of these methods.

To alleviate this limitation, Text2CAD [36] and CADTranslator [46] generate CAD construction sequences from concise and abstract textual descriptions. However, these methods rely on end-to-end, black-box generative models that produce the entire CAD construction sequence in one pass. As a result, the generation process lacks transparency and offers designers limited controllability. Moreover, the inherent geometric and structural complexity of 3D models makes it difficult for designers to precisely articulate design intent using text alone. Recently, with the success of LLMs in code generation, methods such as Query2CAD [5] and CADCodeVerify [2] directly generate CAD software scripts or CADQuery code from text prompts with LLMs, and refine the outputs through feedback from VLMs, which compare the generated CAD models against reference CAD images. While this paradigm improves controllability to some extent, designers can still influence the results only indirectly—typically through images that represent the final CAD form. In this work, we tackle these challenges by leveraging prompt recommendation, multi-modal intent expression and a stepwise

generation method, enabling designers to intervene at each generated CAD modeling step, directly convey their intent, and exert finer control over the outcomes.

2.3 Design Intent Alignment between Human and GenAI

Accurately conveying design intent to GenAI systems is a key challenge faced by designers in human-AI co-creation, and it is also a critical factor influencing the quality of the generated designs [26, 77]. Prior HCI research explores several approaches to help designers express vague ideas in the early stages of conceptual design. Systems such as 3DALL-E [49] and Promptify [9] allow designers to input keywords that represent these vague ideas, and then enrich and clarify the textual input through prompt retrieval or LLM-based prompt generation. AutoSpark [14], BioSpark [32], and GenQuery [73] take this a step further by leveraging such keywords to retrieve and recommend reference images, enabling designers to express their intent by selecting visually relevant examples. In addition, systems such as Inkspire [48] and PromptPaint [19] allow users to sketch geometric features or indicate directions for prompt iteration through drawings. Collectively, these approaches demonstrate that extending expression channels beyond text into 2D visual modalities can effectively concretize and clarify abstract intent in 2D design generation.

In the subsequent stages of iterative design, another critical challenge is conveying localized modification intent and ensuring that generative models interpret it accurately. In 2D design, most iterative generation methods combine text prompts with region-based sketching to guide users in specifying both target areas and design directions [19, 73, 97]. In 3D design generation, systems such as GenPara [18] and ReparamCAD [40] allow designers to adjust shape parameters of local components through text prompts or slider controls, thereby transforming 3D meshes or CAD models. However, the flexibility of such editing remains fundamentally constrained by the system's predefined set of parameterizable design variables. HNC [89] further enables designers to directly edit local geometric features of CAD models through graphical interactions such as dragging. Beyond this, some studies [30, 75, 93] explore the use of language, gestures, and natural interactions in VR environments for capturing real-time user intent, demonstrating that multi-modal interaction can enhance the accuracy and efficiency of intent expression. Nevertheless, how such multi-modal signals should be encoded for GenAI to guide conceptual CAD generation and iteration, and whether GenAI can reliably interpret them, remain open questions. Our work makes an initial attempt to explore these directions.

3 Req2CAD

3.1 Design Goals

Building on the above analysis, we identify three underexplored key challenges in leveraging GenAI to support conceptual CAD, aligned with the progression of the design process: (1) insufficient consideration of the initial design problem, (2) limited support for broad exploration of the solution space, and (3) difficulties in accurately expressing design intent in CAD model generation and

iteration. Accordingly, we formulate the following three design goals to guide the design and development of Req2CAD:

- **G1: Establishing the mapping between design problems and parametric CAD solutions.** Req2CAD should assist designers in reasoning from a given design problem to a corresponding CAD solution ideation that are both structurally valid and functionally aligned, while remaining applicable across diverse tasks.
- **G2: Enhance the exploration and synthesis of broad CAD solution spaces.** Req2CAD should facilitate exploration of diverse alternatives by providing mechanisms to organize and compare large sets of candidate solutions from multiple perspectives based on the characteristics of CAD models, thereby helping designers effectively identify and induce representative design patterns within the solution space.
- **G3: Enhancing the expression and interpretation of design intent.** Req2CAD should ensure that designers can clearly express their design intent for novel CAD generation and iteration, and that GenAI can reliably interpret this intent to reduce iteration costs and improve the efficiency of transforming design ideas into parametric CAD models.

Based on these design goals, we designed and developed Req2CAD, a GenAI-powered interactive system that supports novice designers to create conceptual CAD models for the specific design problems in the early stages of product design. In particular, Req2CAD is targeted at products whose structural design is primarily driven by functional requirements and has diverse structural alternatives, such as household items, everyday consumer products, and mechanical tools. This section will introduce the design rationales behind Req2CAD and the user workflow it supports.

3.2 Design Rationale

The design rationales of our system are grounded in classical product design theories, including the systematic design process [57], functional decomposition theory [10, 29, 74], and the Function–Behavior–Structure (FBS) framework [25]. These theories conceptualize design as a progressive reasoning process from an abstract design problem toward concrete product concepts. Designers first analyze a problem and define the overall product function, and then gradually decompose it into a set of basic sub-functions, until each can be associated with one or more feasible structural principles. Each basic sub-function is expressed in a “verb + object” form (e.g., “**support objects**”, “**store energy**”) [29, 74]. After the function decomposition, designers explore multiple possible realizations for each sub-function by linking it to candidate structural components [25]. Through the evaluation, comparison, and integration of these components, designers can ultimately assemble a coherent structural solution that fulfills the original overall functional requirements (G1) [57]. Building on this design rationale, Req2CAD facilitates the conceptual CAD through two key stages: *Function–Structure Reasoning* and *CAD Model Generation*, as shown in Figure 2. The Function–Structure Reasoning stage assist designers to conduct function decomposition and ideate the component

structure solutions for each sub-function, and the CAD Model Generation stage help designers generate and iterate component-level CAD models according the structure solutions.

3.3 Function-Structure Reasoning

3.3.1 Function Decomposition. Figure 3 shows the interface for the Function–Structure Reasoning stage. In the user workflow, a designer can input a design problem (e.g., ‘*Design a mobile phone support*’) into the textbox at the top of the *Function Decomposition Panel* (Figure 3a) and click the *Decompose Functions* button to initiate the design process. Upon receiving the input, Req2CAD employs an LLM to decompose the problem into a set of basic sub-functions, following the above principle of functional decomposition. LLM are required to express these functions in the “verb + object” format (e.g., ‘*ensure stable positioning*’, ‘*provide sliding*’, and ‘*adjust angle*’). Once the decomposition is complete, Req2CAD initializes a tree-structured Design Flowchart within the *Function Decomposition Panel*, where the target product derived from the design problem is placed as the parent node and the basic sub-functions as child nodes. Designers can then review and edit each sub-function node or add new custom sub-function nodes to the flowchart, and ultimately select the basic functions to be addressed in subsequent design process.

3.3.2 Structure Design Exploration and Induction. After completing function decomposition and selection, designers can add one or more basic functions to the *Selected Function Nodes* area of the *Structure Exploration Panel* (Figure 3b) and click the *Get Recommended Cases* button to request CAD component recommendations from Req2CAD. The system retrieves 50 candidate CAD components from our function–structure knowledge base containing 128k+ diverse CAD components and arranges their multi-view composite images on a 2D canvas of the panel according to CAD feature similarity. When a single function is selected, Req2CAD retrieves all relevant design cases from the knowledge base and randomly samples 50 cases for display, with additional cases accessible via the “More” button. For multiple selected functions, design cases are ranked by the number of the corresponding functions they support, with the system prioritizing and displaying those that satisfy multiple functional requirements simultaneously. For visualization and exploration in the 2D canvas, Req2CAD provides two modes: one based on geometric features and the other on topological features. Designers can switch between them using the *Geometry–Topology* toggle. This enables rapid and multi-perspective exploration of the broad design space (G2). Furthermore, based on the 2D layout, Req2CAD clusters recommended cases into five groups, visually distinguished by border colors. This clustering assists designers in efficiently synthesizing representative structural design patterns from large sets of recommendations (G2). Designers can zoom in and out of the canvas to shift between a global overview and detailed local inspection of a specific cluster. For any case of interest, clicking its image opens a pop-up window that displays the associated sub-functions and the text prompts used in its generation. Designers can then add the design case to the design flowchart, where Req2CAD automatically creates a recommended CAD node and links it to the relevant sub-function nodes.

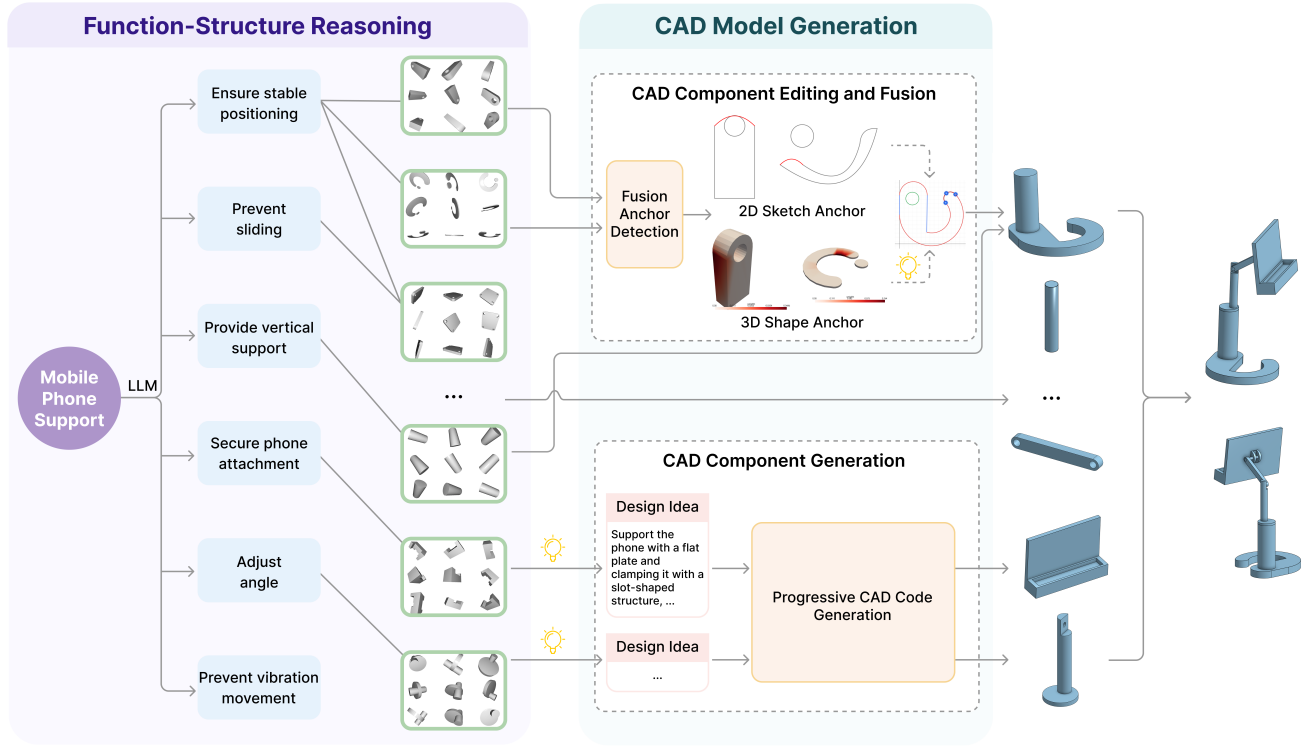


Figure 2: The user workflow with the Req2CAD system.

By systematically exploring relevant design cases for each required function, abstracting typical structural patterns, and filtering candidate solutions, designers can formulate preliminary conceptualizations of component-level structures. The outcomes of each step are recorded in the design flowchart, ensuring traceability of the reasoning process. Afterward, designers can advance to the next “CAD Model Generation” stage via the top of the interface.

3.4 CAD Model Generation

Building on the candidate structures, designers can now develop and refine their ideas. Req2CAD supports this transition through its LLM-based progressive CAD generation capabilities, enabling designers to transform abstract concepts into detailed parametric CAD components (G3). To further support refinement, Req2CAD also provides a graphical CAD component editor, allowing designers to fine-tune both newly generated and retrieved CAD models. Figures 4 and 5 illustrate the interfaces supporting these two complementary functionalities.

3.4.1 Progressive CAD Component Generation. As shown in Figure 4, the design flowchart is preserved on the left. Designers can click on any recommended design node to view the text prompts associated with its generation. By referencing these designs and prompts, they can compose their own textual descriptions of design concepts in the input box of the *Chat Panel* (Figure 4b). Req2CAD then employs an LLM to generate CAD construction code in CADQuery,

which is a Python-based parametric CAD language with a built-in compiler that interprets code and renders parametric 3D objects. The generated code appears in the *Code Editing Panel* (Figure 4c), where designers can review or modify it. By clicking the *Run Code* button, the CAD component will be rendered and displayed in the *Render View* of the *Visualization Panel* (Figure 4d).

For complex CAD components, Req2CAD supports step-by-step incremental generation as shown in Figure 7. Designers can begin with a simple primitive (e.g., ‘generate a cube’). Beyond rendering the generated CAD model, Req2CAD parses its B-rep into a graph structure where each face is represented as a node and adjacencies between faces as edges, and visualize this graph in the *B-rep Network* area with colors matching its corresponding face in the *CAD Render View*. When additional structures need to be added, the designer can switch to iteration mode by clicking the *Modify* button. Designers can first select a target face by clicking its corresponding node in the B-rep graph, with the faces highlighted in the *Render View*. To avoid occlusion, Req2CAD provides an interactive slider above the render view that controls the separation distance between faces to generate an exploded view of the CAD model. Once a target face is selected, the designer can enter a textual description of the intended operation (e.g., ‘cut a rectangular slot’) into the text box. Req2CAD then extends the previously generated CAD code to implement the iterative modification. This process

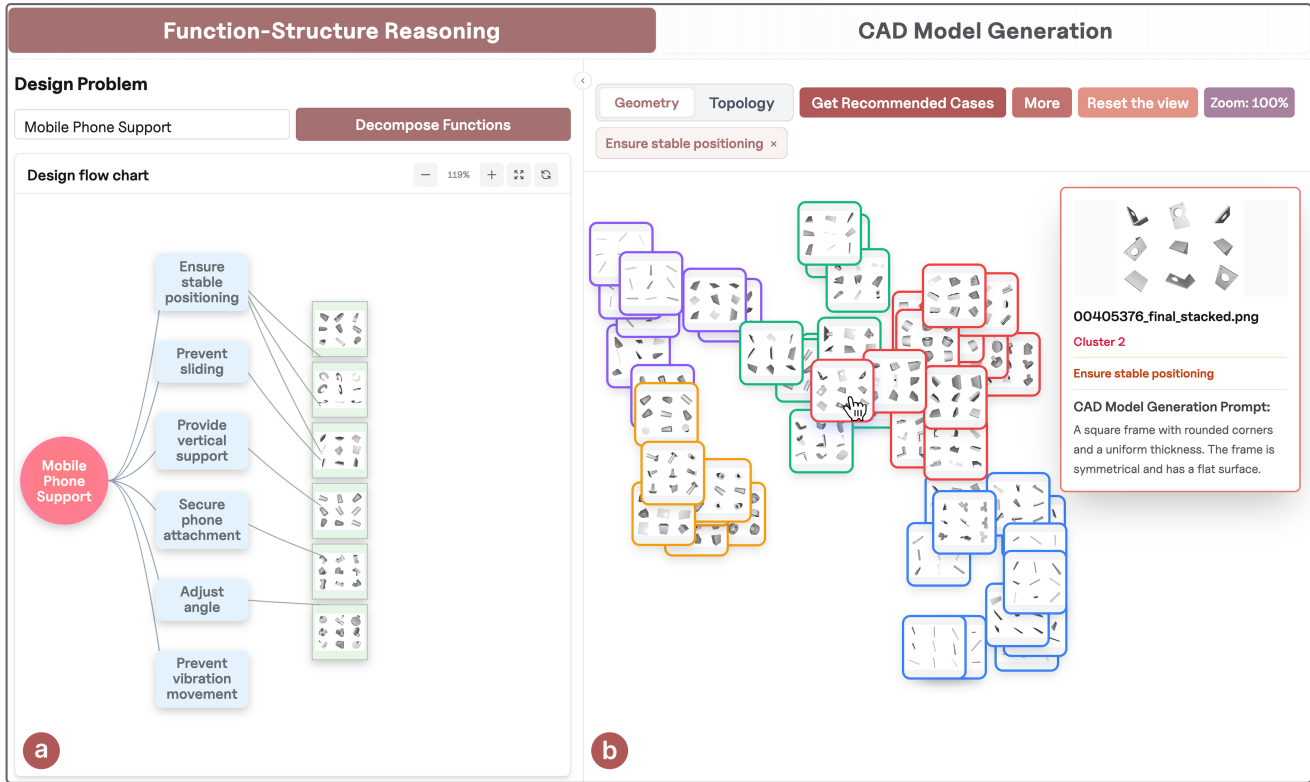


Figure 3: Interface of the Function-Structure Reasoning stage. It consists of (a) Function Decomposition Panel for decomposing basic sub-functions from the design problem; (b) Structure Exploration Panel for exploring the design space and inducing the representative structure pattern corresponding to one or more basic sub-functions.

can be repeated until the component reaches its final form. By decomposing the creation of complex CAD components into multiple incremental steps, Req2CAD reduces the burden of writing precise prompts while giving designers finer control over each iteration. The graphical face-selection mechanism further ensures accurate specification of spatial regions, overcoming text-based limitations.

Generated components can be added to the design flowchart via the *Add to Design Flow* button and linked to functional nodes. Additionally, the *File Operations* area in the Visualization Panel allows designers to either download the component as a STEP file or upload it directly to an Onshape project for subsequent assembly.

3.4.2 CAD Component Editing and Fusion. To support designers in refining those recommended components from the knowledge base, Req2CAD also provides a graphical CAD component editor. Designers can switch between the CAD generator and editor by clicking the buttons in the upper-left corner of the *CAD Creation* page. The editor primarily consists of a 2D sketch canvas (Figure 5b) and a slider to control 3D extrusion distance (Figure 5c). Designers can import a component from the design flowchart for fine-tuning. Req2CAD parses the component's initial 2D sketch and extrusion parameters and visualizes them on the 2D canvas and 3D render view. Designers can modify the component by dragging sketch

points, adjusting the extrusion slider, or editing parameter values in the *Parameter Editing Panel* (Figure 5e). The revised CAD component can then be added back to the design flowchart as an iterative CAD node, recording its evolution in the design process.

During component iteration, following design simplification and modularization principles [57, 81], designers can optimize not only individual components but also fuse components with similar functions or structures to reduce complexity and eliminate redundancy. To support and inspire designers in exploring component fusion, Req2CAD allows designers to select two CAD components from the design flowchart and add them to the *Component Fusion Panel* (Figure 5d). The system applies our proposed fusion anchor detection algorithm to the 2D sketches and 3D meshes of the two components, highlighting regions with similar geometric features in red. Designers can click the circular markers below to view multiple sets of matched regions. These similarity regions serve as fusion anchors, stimulating designers' conceptualization, which can then be further implemented in the CAD editor or generator.

After all components are finalized, they can be exported to Onshape or other CAD software for assembly, producing a parametric CAD model that fulfills the overall function requirements within the given design problem. More details about how Req2CAD leverages

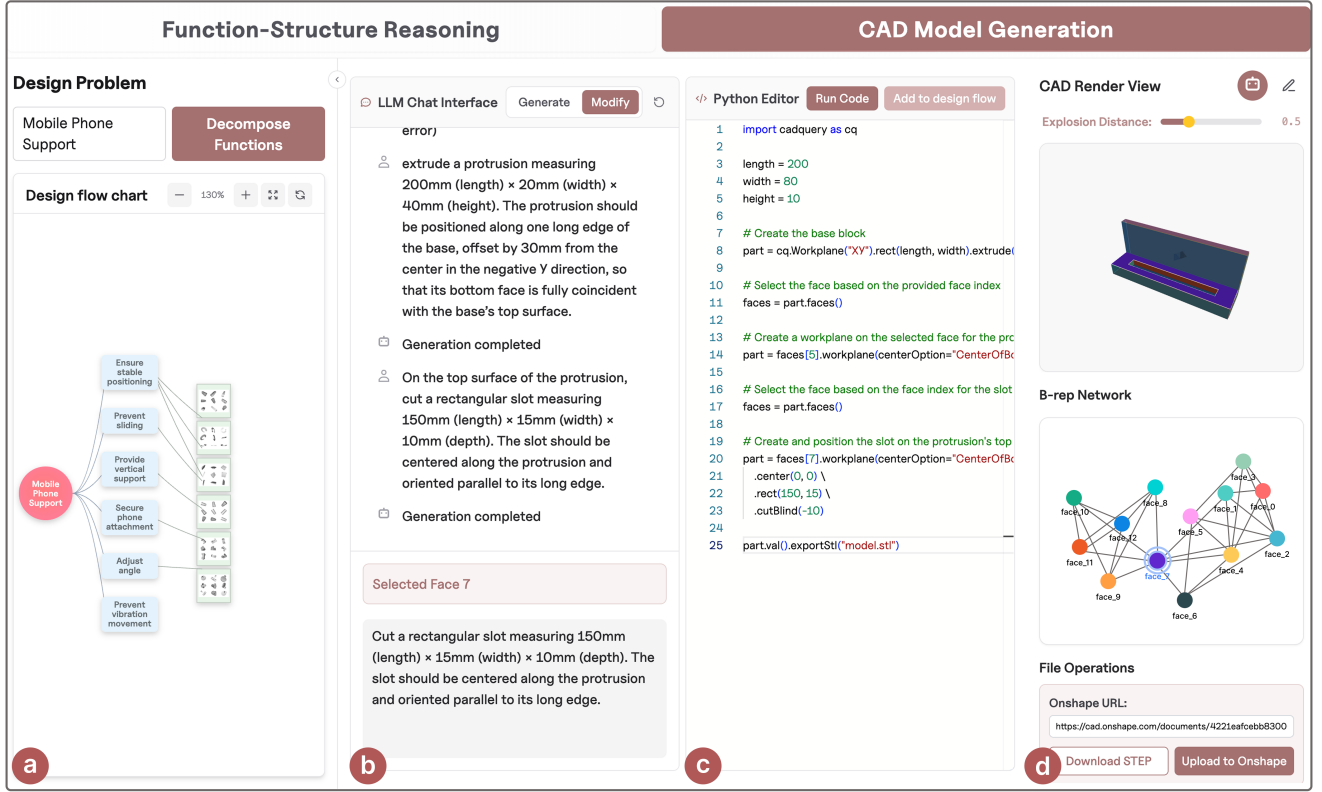


Figure 4: Interface of the progressive CAD component generation for the CAD Model Generation stage. It consists of (a) Function Decomposition Panel; (b) Chat Panel for conversations with LLM; (c) Code Editing Panel for reviewing and running the generated CADQuery code; (d) Visualization Panel for rendering and interacting the generated CAD model.

LLMs for function decomposition and CADQuery code generation are provided in Section 4.1.1, 4.3, and the supplementary material.

4 Technical Details

Req2CAD was built as a web-based system with a Next.js¹-based front-end and a python-based back-end. Communication between the front-end and back-end is facilitated by an API built using the Flask² framework. React Konva³ and Three.js⁴ are employed to implement the 2D sketch editing canvas and CAD model rendering view, respectively. GPT-4o [55] is employed as the underlying LLM to decompose basic sub-functions and generate CADQuery code. More technical details will be introduced in this section.

4.1 Function-Structure Knowledge Base

4.1.1 Function Keywords Annotation. To establish a mapping from functions to structural design space, we built a function-structure knowledge base on the Text2CAD dataset [36]. Text2CAD extends DeepCAD [88] with text descriptions and contains 175k CAD components, each with its B-rep, construction sequences, four levels of

text prompts (from abstract-level to expert-level), and tiled rendering images from nine viewpoints. All CAD components were collected from real design projects created by designers on the Onshape public collection, covering a wide range of small- to medium-scale mechanical parts, such as brackets, gears, connectors, housings and other components for product design, mechanical design, and prototyping. The dataset does not include architectural structures, large-scale assemblies, or heavy machinery, as such objects are generally not represented among the sketch-and-extrusion-based models selected from DeepCAD dataset. This scope aligns with the design goal of our system, which is to provide component-level structure design references that support the realization of specific sub-functions. A major limitation of Text2CAD is the absence of functional annotations. To address this, we leveraged GenAI techniques to enrich each component with function labels. First, the multi-view rendering image of a CAD component was input into a VLM, Qwen2.5-VL-7B-Instruct [6], and then we prompted it to act as a professional engineering designer and describe all possible functions of the component (e.g., *'Distributing Load: It can help distribute a load over a larger area, reducing stress concentration on a smaller contact point.'*). When a function could not be inferred, the model was required to return *"It is difficult to determine the function of this component."* Then an LLM, Qwen3-14B [91], was

¹<https://nextjs.org/>

²<https://flask.palletsprojects.com>

³<https://konvajs.org/>

⁴<https://threejs.org/>

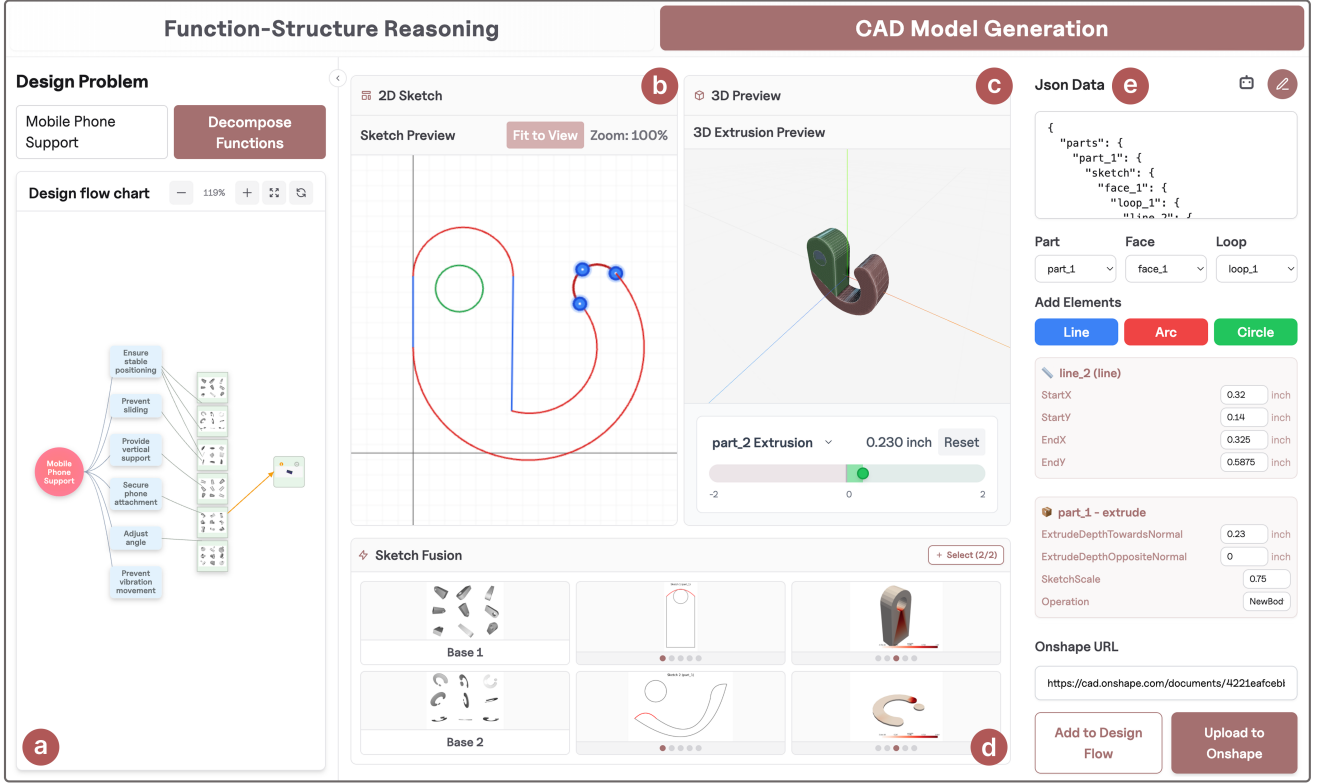


Figure 5: Interface of the CAD component editor for the CAD Model Generation stage. It consists of (a) Function Decomposition Panel; (b) 2D Sketch Canvas for editing sketch; (c) 3D Rendering View for controlling extrusion distance and preview of the rendered CAD model; (d) Component Fusion Panel for identifying and displaying potential fusion anchor; (e) Parameter Editing Panel for editing CAD model with specific parameter values.

employed to extract function keywords in "verb + object" form (e.g., "distribute load") from these descriptions. Keywords were returned in a nested JSON list. As a result, each CAD component is annotated with both a natural language description and a set of function keywords, forming a mapping from functions to structure design solutions. After filtering out indeterminate cases, we obtained a function-structure knowledge base containing 128, 873 CAD components. The VLM annotation process required about 8 days, while keyword extraction took 12 hours. Prompt templates used in this pipeline are provided in the supplementary material. We have also made this dataset publicly available⁵ to promote future research on learning-based CAD designs.

4.1.2 Function-Driven Design Case Retrieval. To map sub-functions to function keywords in the knowledge base, a text embedding model, Qwen3-Embedding-4B [99], is leveraged to compute embedding vectors for both the user-selected basic function f^q and all function keywords $\{f_i^k | f_i^k \in F\}$ in the knowledge base, and cosine

similarity is calculated between them as:

$$\text{sim}(f^q, f_i^k) = \frac{\text{emb}(f^q) \cdot \text{emb}(f_i^k)}{\|\text{emb}(f^q)\| \|\text{emb}(f_i^k)\|}. \quad (1)$$

Function keywords $\{f_i^k\}$ with similarity scores above **0.7** are considered relevant, and their associated CAD components $\{c_i\}$ are retrieved to form a candidate pool. When the user requests design recommendations, Req2CAD randomly samples 50 components from this pool for display. For multiple sub-function queries, the system first identifies relevant keywords for each sub-function using the same method. It then ranks all associated CAD components by the number of matched functions, and prioritizes returning the top 50 design cases which satisfy multiple targets. Along with the retrieved CAD components, Req2CAD also returns their text generation prompts, providing designers with references for composing effective LLM-based CAD code generation prompts.

4.2 Dual-feature CAD Representation

To support rapid preview and pattern recognition of retrieved CAD components, we introduce a dual-feature representation, as shown in Figure 6: a B-rep-based graph captures topological structures, while point cloud embeddings represent geometric shapes. Building

⁵<https://huggingface.co/datasets/QianzhiJing/Req2CAD>

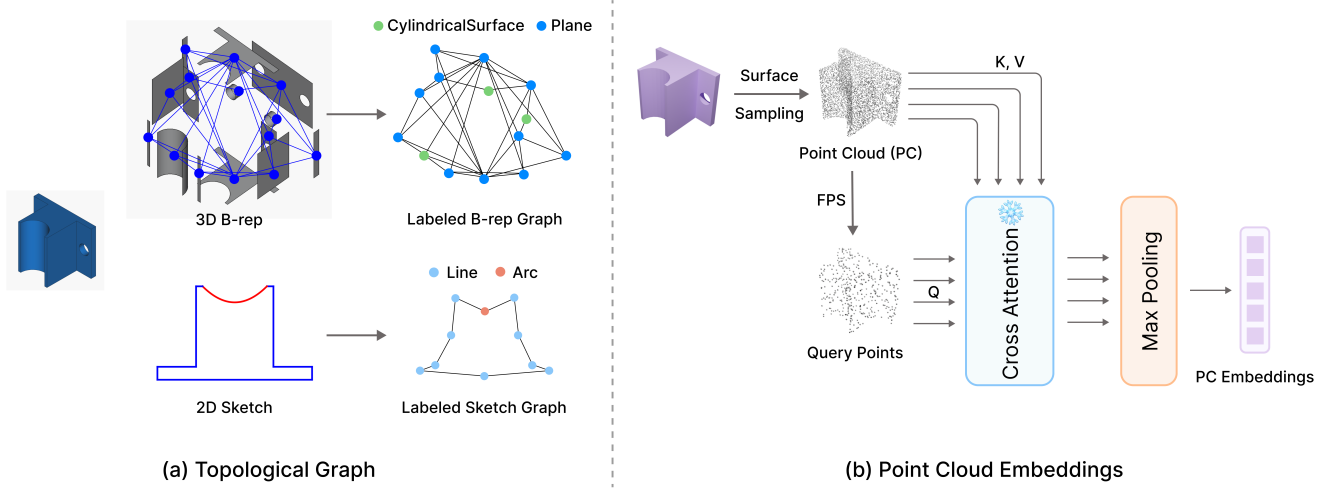


Figure 6: Dual-feature CAD Representation. It consists of (a) B-rep graph for representing topological features, and (b) Point cloud embeddings for representing geometric features.

on these representations, we further develop corresponding similarity measures for topology and geometry, which enable similarity-based 2D arrangement of components and detection of analogous regions, thereby facilitating exploration and design fusion from multiple perspectives.

4.2.1 B-rep Graph. As shown in Figure 6a, given a CAD component, its B-rep is extracted from the corresponding STEP file. We then construct a graph $G = (V, E)$, where each vertex $v \in V$ represents a face in the B-rep, and is assigned a label l represents the face type (e.g., *plane*, *cylindrical surface*, *conical surface*). An edge $(v_i, v_j) \in E$ is created if the corresponding faces are adjacent in the B-rep structure. In this way, the topology of a CAD component is formalized as a labeled graph. To measure the similarity between CAD components represented as graphs, we adopt the Weisfeiler–Lehman (WL) sub-tree kernel $k_{WL}(\cdot, \cdot)$, which is effective for labeled graph comparison. Given two graphs G_i and G_j , the kernel value is defined as:

$$k_{WL}(G_i, G_j) = \langle \phi_{WL}(G_i), \phi_{WL}(G_j) \rangle, \quad (2)$$

where $\phi_{WL}(G_i)$ denotes the feature vector extracted from the iterative relabeling of the WL algorithm. This yields a positive semi-definite kernel matrix $K \in \mathbb{R}^{N \times N}$, where $K_{ij} = k_{WL}(G_i, G_j)$, $i, j \in \{1, \dots, N\}$. The pairwise distance matrix D is then derived as

$$D_{ij} = \sqrt{K_{ii} + K_{jj} - 2K_{ij}}, \quad (3)$$

which encodes the topological similarity among CAD components. Then we perform dimensionality reduction on the distance matrix D using t-SNE [51]. Each CAD component is thus projected into a 2D coordinate such that topologically similar designs are positioned closer together, supporting effective clustering, retrieval, and design space exploration.

In addition to 3D B-rep topology, we adopt a similar approach to represent CAD components' 2D sketches as graph structures. For a given sketch, we construct a graph $G^s = (V^s, E^s)$, where each vertex $v \in V^s$ represents a 2D geometric element with a type label

indicating its category, including *Line*, *Arc*, and *Circle*. An edge $(v_i^s, v_j^s) \in E^s$ is created if two geometric elements are connected through endpoints in the sketch. We then perform Maximum Common Subgraph (MCS) detection to identify the largest overlapping labeled subgraph. The resulting MCS highlights recurring geometric motifs shared between the two sketches, such as line–arc junctions or repeated circular structures. These shared subgraphs serve as 2D sketch fusion anchors, providing structure correspondences that support meaningful design integration and recombination across CAD components.

4.2.2 Point Cloud Embeddings. To capture the geometric features of CAD components, we adopt a point cloud embedding method based on 3DShape2Vec [95] and use its pre-trained model on large-scale 3D dataset ShapeNet [11]. As shown in Figure 6b, for a given CAD component, we first uniformly sample n points from its mesh surface, and then apply Farthest Point Sampling to select m representative points. These points are spatially well-distributed, serving as local centers and query points for constructing local geometric features. Based on 3DShape2Vec, We employ the cross-attention mechanism [83] to model the relationships between each query point q_i and all sampled points $\{x_j\}_{j=1}^n$ as:

$$\text{Attn}(q_i, X) = \sum_{j=1}^n \alpha_{ij} W_v x_j, \quad (4)$$

$$\alpha_{ij} = \frac{\exp\left(\frac{(W_q q_i)^T (W_k x_j)}{\sqrt{d}}\right)}{\sum_{t=1}^n \exp\left(\frac{(W_q q_i)^T (W_k x_t)}{\sqrt{d}}\right)}, \quad (5)$$

where W_q, W_k, W_v are the linear projections for queries, keys, and values, respectively, and α_{ij} denotes the attention weight of the j -th sampled point for query point q_i . Due to spatial proximity, points closer to the query naturally receive higher attention weights, allowing the output feature to effectively encode the local geometry around. Compared to MLP-based methods like PointNet [62] and

CNN-based methods like EdgeConv [85], our approach overcomes the limitations of hierarchical aggregation by directly modeling pairwise relationships within the attention field, reducing information loss and more accurately capturing fine-grained local geometry. Next, we aggregate all local features via max pooling to obtain a D -dimensional global embedding representing the overall geometric shape of the CAD component. For a set of retrieved CAD components, these embeddings can be projected into 2D space using t-SNE, enabling visualization and layout based on geometric similarity.

For the locally similar 3D regions detection between two CAD components, we compute the m local embeddings for each component and construct an (m, m) cosine similarity matrix. The top- k most similar pairs of local regions are selected as candidate matching fragments. For intuitive visualization, we leverage the cross-attention weights α_{ij} to generate a heatmap on the mesh surface: points with higher weights contribute more to the local feature and are rendered with deeper colors (Figure 5d). This visualization not only highlights the spatial contribution of each point to local features but also enhances the interpretability of the detected geometrically similar regions.

4.3 Progressive CAD Code Generation Method

Instead of requiring designers to provide a complete prompt for an entire CAD model like previous methods [2, 36, 41], our system decomposes the modeling process into multiple incremental steps and enables designers to directly control the generation at each step through text prompts and graphic interactions, as shown in Figure 7. Given an initial text description of the target 3D object, x_0 , we first prompt an LLM to generate initial CAD code, y_0 , which can be formulated as:

$$y_0 \sim P_{LM}(y_0 \mid x_0). \quad (6)$$

The code is executed by the CADQuery compiler ψ , yielding the initial parametric CAD component $d_0 = \psi(y_0)$. Each component d_t consists of a set of faces $\{f_1^t, f_2^t, \dots, f_{n_t}^t\}$. A key property of CADQuery, inherited from the OpenCascade⁶ kernel, is that the ordering of faces is determined by the underlying OpenCascade topology rather than being random; as long as the modeling history y_t remains unchanged, the index of each face is stable across executions: $\forall i, f_i^t \equiv f_i^{t'}$ if $y_t = y_{t'}$. Leveraging this stability, we present faces and their indices to the user in the form of a B-rep graph structure, and determine the index of the face to be modified based on the vertex selected by the user. In subsequent iterations, the designer provides an additional prompt x_{t+1} , together with the index k of a target face in d_t . The language model generates an incremental code fragment, Δy_{t+1} , conditioned on the prior code and the specified face index:

$$\Delta y_{t+1} \sim P_{LM}(\Delta y_{t+1} \mid x_{t+1}, y_t, k). \quad (7)$$

The new CAD code is obtained by concatenation,

$$y_{t+1} = y_t \oplus \Delta y_{t+1}, \quad (8)$$

and executed to produce the updated component $d_{t+1} = \psi(y_{t+1})$. By repeating this procedure, a sequence of designs is constructed: $(x_0, y_0, d_0) \rightarrow (x_1, y_1, d_1) \rightarrow \dots \rightarrow (x_T, y_T, d_T)$, where each step refines the previous design through local face-index-based

operations. Finally, a CAD component with a complex structure is created.

5 Technical Evaluation

5.1 Function Keywords Annotation

We randomly sampled 50 CAD components from the annotated knowledge base and recruited two designers (Designer 1 with six years of work experience and Designer 2 with three years of training in mechanical design). Each designer was asked to provide function keywords for 25 randomly chosen CAD components based on their multi-view images. This task took approximately one hour per designer. Subsequently, two experts (each with over five years of experience) evaluated both the outputs of our GenAI-based automatic annotation pipeline and the designers' manual annotations. The evaluation involved counting the total number of functions, verifying correctness, and recording repetitions. After independent evaluation, the experts discussed any discrepancies to reach consensus. Table 2 reports the mean and standard deviation of the results. Our automatic annotation pipeline identified slightly more functions than the designers, with a moderate to high overlap between the two, but the accuracy of GenAI pipeline was somewhat lower than manual annotation. Nevertheless, this pipeline maintained an accuracy above 70% while being far more efficient than manual annotation. Figure 8 illustrates representative annotation examples of GenAI and designer, with overlapping functions (as determined by the experts) connected by dashed lines. As shown, the designers' annotations were typically more context-specific and employed more technical terminology (e.g., "*Limit the axial position*," "*Support the rotating axis*"), whereas the GenAI annotations tended to be more general and basic (e.g., "*align parts*," "*support component*"). Moreover, the function descriptions generated by the VLM demonstrate its ability to systematically infer functions from local geometric features of CAD components. These findings indicate that our automatic annotation pipeline provides reliable basic function annotations, alleviating the time- and labor-intensive burden of large-scale manual labeling. More annotation examples are provided in the supplementary material.

5.2 CAD Code Generation

We designed two quantitative experiments to evaluate the effectiveness of our progressive CAD generation method. In the first experiment, we compared two mainstream approaches for using LLMs to generate CAD models. For Method I (Minimal JSON), we use an LLM to generate CAD modeling parameters and construction sequences in a predefined format (i.e., the minimal JSON format defined in Text2CAD [36]). Specifically, given a CAD model prompt (e.g., "*create a cylindrical object with a height of 0.09494 units and an outer diameter of 1.5 units*"), the model was asked to output key parameters of lines, arcs, and circles in a 2D sketch, along with the 3D extrusion distance based on that sketch, and arranges these operations in sequence. The outputs were then parsed and converted into STL files. For Method II (CADQuery), we ask the LLM to generate CADQuery Python code from the same CAD model prompt, and exporting STL files by executing the generated code. The test dataset was drawn from CADCodeVerify [2], which contains 200 CAD generation samples, each consisting of text prompts

⁶<https://www.opencascade.com/>

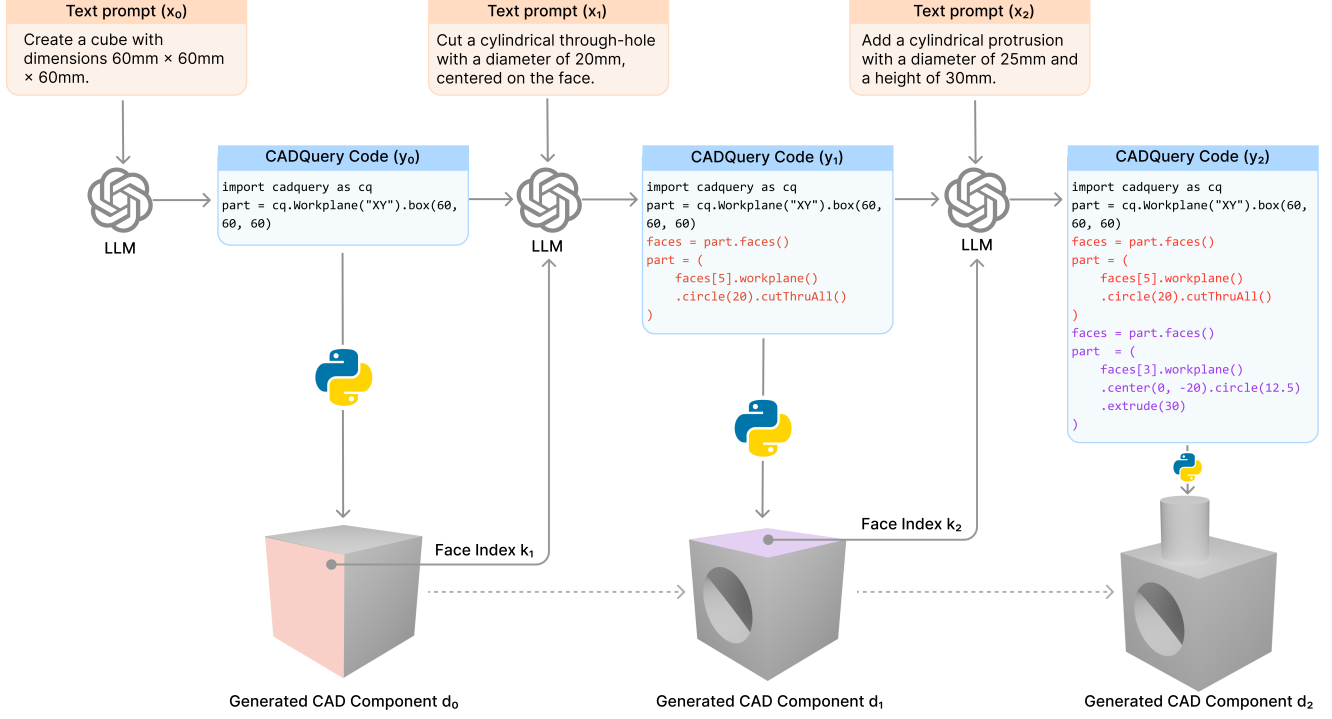


Figure 7: Example for generating CAD component using our progressive CAD code generation method.

Table 2: Annotation results of function keywords based on multi-view images of 50 CAD components, including the average number and percentage of correctly predicted functions, and the average number of repetitions from our GenAI-based annotation pipeline and designers.

		Number (#)	Accuracy (%)	Number of Repetitions
Sample 1-25	Designer 1	3.11 ± 0.74	92.36 ± 16.83	2.28 ± 0.87
	Ours	3.61 ± 0.68	74.10 ± 23.79	
Sample 26-50	Designer 2	3.44 ± 0.90	92.67 ± 14.09	2 ± 0.67
	Ours	3.61 ± 1.01	72.04 ± 34.25	

Table 3: Quantitative evaluation for CAD model generation between sequence-based generation method (Minimal JSON) and one-step code-based generation method (CADQuery) on 100 samples.

	Point Cloud Distance↓	Hausdorff Distance↓	Compile Rate↑
Minimal JSON	0.204 ± 0.515	0.300 ± 0.504	90%
CADQuery	0.149 ± 0.439	0.229 ± 0.433	89%

with specific numerical descriptions and corresponding CAD STL files. We randomly selected 100 samples from it. Experiments were conducted on GPT-4o [55]. Following the evaluation method proposed in CADCodeVerify, we aligned the orientation of generated results with the ground truth, normalized them to a unit cube, and computed two geometric similarity metrics to measure the distance between generated models and ground truth: *Point Cloud Distance*

and *Hausdorff Distance*. We also reported the percentage of successfully compiled code as the *Compile Rate*. For failed compilations, we assigned a distance value of $\sqrt{3}$ (the maximum possible distance between two points in a unit cube) to uniformly penalize unsuccessful generations. Table 3 shows the quantitative results. Both methods achieved comparable compile rates, while CADQuery-based generation method can produce models with the higher geometric fidelity,

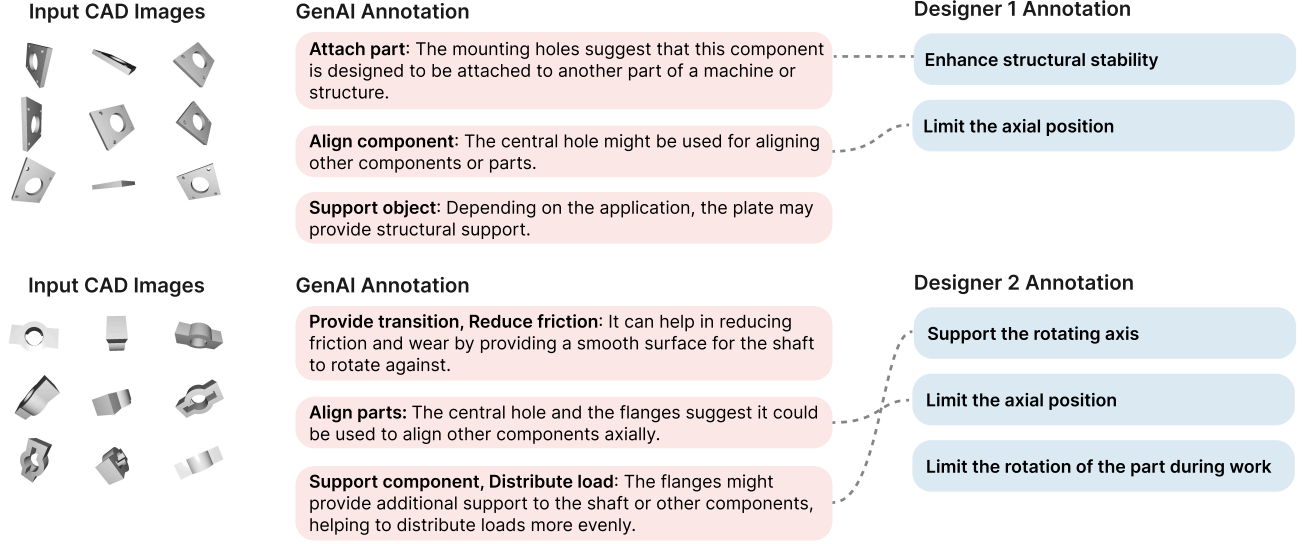


Figure 8: Examples for the function annotation results provided by our proposed GenAI-based annotation pipeline and two designers. In the GenAI output, the regular text shows the function descriptions generated by the VLM, while the bold text shows the function keywords extracted by the LLM.

Table 4: Quantitative evaluation for code-based CAD model generation between one-step generation method and our proposed progressive generation method on 20 complex samples.

	Point Cloud Distance↓	Hausdorff Distance↓	Compile Rate↑
One-step Generation	0.197 ± 0.513	0.262 ± 0.502	90%
Progressive Generation (Ours)	0.013 ± 0.011	0.073 ± 0.080	100%

more closely matching the ground truth samples. This may be attributed to the fact that the LLM has been trained on large-scale Python code, enabling it to better learn and generate syntactically and logically consistent CADQuery code; in contrast, minimalJSON is an artificially defined format with little prior exposure, making the models more prone to format errors or semantic omissions.

In the second experiment, we compared the above best-performing method (GPT-4o generating CADQuery code in one step) with our proposed progressive generation method. We randomly selected 20 complex CAD samples for test from CADCodeVerify dataset that require multiple extrusion steps, and manually segmented each original prompt into stepwise sub-prompts for progressive generation. The evaluation metrics remained the same as in the first experiment. The results in Table 4 show that our progressive generation method significantly reduced the distance between generated models and ground truth, greatly improving consistency with input prompts. This demonstrates that our method is better suited for generating CAD models that align with designers' intended goals.

6 User Study

To evaluate the effectiveness of Req2CAD in assisting designers during the CAD model design process, compared with the conventional workflows and existing GenAI tools designers currently used in practice, we conducted a within-subject user study and explored the following research questions:

- RQ1: Can Req2CAD assist designers in creating CAD models that better satisfy the functional requirements of the design problems and achieve higher overall quality?
- RQ2: Can Req2CAD effectively support design ideation and CAD creation as well as enhance human-AI co-creation experience?
- RQ3: How does Req2CAD influence designers' ideation and design processes when collaborating with AI?

6.1 Method

Participants. We recruited twelve novice designers with experience using CAD software through a university social networking site (four females, eight males; aged 20 to 28 $M = 22.33$, $SD = 2.32$). Their academic backgrounds were primarily in Industrial Design,

Table 5: Overview of participant details in the user study, with gender, age, discipline, occupancy, years of CAD experience, and primary uses of GenAI in design activities.

ID	Gender	Age	Discipline	Occupation	CAD Exp. (y)	Use of LLMs
P1	Male	21	Mechanical Engineering	Student / BA	3	RA
P2	Female	21	Mechanical Engineering	Student / BA	3	GD
P3	Male	28	Artificial Intelligence	Student / PhD	5	RA, GD, GCC
P4	Male	25	Industrial Design	Student / PhD	7	RA, GD
P5	Female	22	Architecture	Student / BA	3	GD
P6	Male	20	Mechanical Engineering	Student / BA	2	RA, CSG
P7	Male	21	Artificial Intelligence	Student / BA	0.5	GD, GCC
P8	Female	24	Industrial Design	Student / PhD	2	RA, GD, CSG
P9	Female	24	Industrial Design	Student / MA	3	RA, GD
P10	Male	20	Mechanical Engineering	Student / BA	1.5	RA, GD, CSG
P11	Male	21	Mechanical Engineering	Student / BA	0.5	-
P12	Male	21	Artificial Intelligence	Student / BA	0.5	RA, GCC

Note:

RA: requirement analysis; GD: generating design descriptions; GCC: generating CAD creation code or scripts; CSG: CAD software operation guidance; -: Never Used in design.

**Figure 9: The process of participants engaging in the user study.**

Mechanical Engineering, and Architecture (Table 5). On average, participants had 2.96 years ($SD = 2.40$) of study experience in design-related disciplines and 2.58 years ($SD = 1.85$) of CAD software experience. All had prior experience with GenAI (e.g., LLMs, text-to-image models, and text-to-3D models), and eleven designers reported using GenAI in their design work, primarily for analyzing requirements and generating design concepts in textual or visual form, while six participants have used LLMs for CAD software operation guidance or generating CAD construction code. Each participant received 80 RMB per hour as compensation.

Settings. Since our system is designed to support designers in conceptual CAD ideation and component creation, serving as an assistive tool rather than a replacement for CAD software to enhance the design process, our setup for experimental group consisted of Req2CAD and Onshape. Designers were instructed to use Req2CAD for early-stage ideation and CAD component creation, and Onshape for component assembly for further refinement. For the baseline, we adopted ChatGPT and Onshape. This choice was motivated by the fact that ChatGPT provides access to GPT-4o, the underlying LLM used in our system, while also offering text generation and text-to-image generation capabilities frequently used by designers in real-world practice. To better simulate authentic design scenarios, participants using the baseline system were allowed to freely interact with ChatGPT, including tasks such as requirements analysis, design concept generation, product image creation, Onshape usage guidance, and CAD code generation. They were also permitted to leverage the internet or personal knowledge bases for design case retrieval, while component creation and assembly were conducted in Onshape.

Procedure and Tasks. Figure 9 illustrates the procedure for each participant. Each participant was required to undertake two design tasks, performing one with the experimental system and the other with the baseline system. The assignment of tasks to systems and the order of system use were counterbalanced across participants. Participants were asked to produce a conceptual CAD model addressing the given design problem in each task. Upon completion, participants submitted a CAD model and, when necessary, an accompanying written description. Task I was “Design a device for holding a mobile phone on a flat surface”, and Task II was “Design a device for hanging towels in a bathroom”. The study began with a briefing and background questionnaire, followed by two rounds of design tasks. In each round, participants first received a 10-minute training covering system usage, basic prompt-writing techniques, and an introduction of the design task. Then participants had 40 minutes to perform design ideation and create a CAD model with the support of the assigned system. Upon completing both tasks, participants were invited to complete questionnaires and a semi-structured interview to provide feedback on how each system supported their design activities and shaped their human-AI interaction experience. The study was conducted in person (face-to-face), with each participant using their own personal computer, and all sessions were screen-recorded for later analysis (RQ3).

Measures. Building on the design goals of our system and prior work on human-AI collaboration in 3D design [18, 26, 49], the questionnaire administered to participants included seven measures to collect feedback on the effectiveness of our system in supporting conceptual CAD and enhancing the design experience (RQ2, RQ3): *Exploration* (the system helped me broadly explore the design space

relevant to the design problem), *Intent Expression* (I was able to accurately convey my design intentions when using the system), *Match* (the outputs of the system aligned with my input requirements), *Satisfaction* (I am satisfied with the final design solutions produced in collaboration with the system), *Transparency* (the design process supported by the system was transparent, allowing me to track and manage it), *Ease of Use* (the system was easy to use), and *Usefulness* (the system was helpful in assisting my design tasks). Participants rated these measures on a 5-point Likert scale (1 = strongly disagree, 5 = strongly agree). The questionnaire also included three measures adapted from NASA-TLX [28] to assess task load on a 7-point scale: Mental Demand, Effort, and Frustration. To further investigate how each system supported design ideation (RQ3), participants were asked to report the sources of inspiration for their final design outcomes, with system-specific options detailed in Figure 13. Since final design solutions often consisted of multiple CAD components that could originate from different sources, participants were allowed to select multiple options. After collecting all 24 design outcomes, four experts with more than five years of product design experience in industry were invited to evaluate the quality of the designs in terms of *Functionality*, *Feasibility*, and *Novelty* [57, 68, 81] (RQ1). Detailed definitions of these measures are provided in the supplementary material. Experts reviewed the Onshape project files containing the final CAD models, along with necessary textual descriptions, and rated them on a 5-point Likert scale. For participant questionnaire measures, normality of the difference scores was first assessed using the Shapiro–Wilk test; when the assumption of normality held, paired-sample t-tests were conducted, otherwise the Wilcoxon signed-rank test was applied. For expert evaluations, normality within each group was examined using the Shapiro–Wilk test; when both groups met the normality assumption, independent-sample t-tests were conducted, otherwise the Mann–Whitney U test was applied.

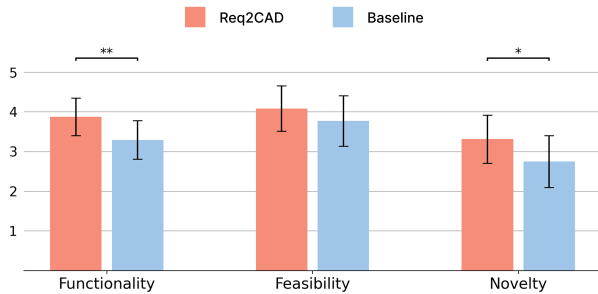


Figure 10: Expert evaluation results of the design schemes produced in the User Study for Req2CAD and baseline condition. (Error bars represent standard deviation. *: $p < 0.05$, **: $p < 0.01$.)

6.2 Evaluation Results

RQ1: Design Solution Quality Evaluation. Figure 10 shows experts' assessments of the CAD models created by participants using Req2CAD and the baseline system. It demonstrates that designs

produced with Req2CAD perform better in terms of functionality (Req2CAD : $M = 3.88, SD = 0.47$ /Baseline : $M = 3.29, SD = 0.49$ /Independent t -test, $t = 2.85, p = 0.009$), feasibility (Req2CAD : $M = 4.08, SD = 0.57$ /Baseline : $M = 3.77, SD = 0.63$ /Independent t -test, $t = 1.22, p = 0.24$), and novelty (Req2CAD : $M = 3.31, SD = 0.60$ /Baseline : $M = 2.75, SD = 0.65$ /Mann–Whitney U , $U = 108.5, p = 0.034$), with statistically significant improvements in functionality and novelty. The absence of a significant difference in feasibility may be due to experts' generally high ratings for all designs on this metric. Figure 11 illustrates some representative design examples. Req2CAD's results demonstrate stronger functional extensibility. In the mobile phone holder task, for instance, Req2CAD's designs not only support the phone but also enable angle adjustment (column 1 and 2 in Task I), enhancing usability. In the towel hanger task, Req2CAD's designs incorporate multiple rods and hooks (column 3 in Task II), and the design in column 2 of Task II further adds a flat panel structure on top to accommodate additional items, addressing more diverse user needs. By contrast, baseline system designs tend to remain limited to "basic support" or "fixed hooks". Req2CAD's designs also display greater diversity in realizing similar functions. For example, to enable angle adjustment in phone holders, the design in column 1 uses a ball joint, whereas column 2 applies a hinge joint. Similarly, for multi-hook towel hangers (column 3 in Task II), the baseline system repeats identical hook structures, while Req2CAD combines rods, flat hooks, and slender hooks, demonstrating the ability of Req2CAD to support designers in exploring a broader design space related to functionality and diverse design variations, thereby overcoming individual design fixation.

In terms of feasibility, Req2CAD's designs exhibit stronger structural soundness and practical realizability. The design in column 1 in Task II, for example, reinforces the connection between the towel hanger and the wall through screw holes and a thickened mounting plate, improving stability and durability. In contrast, the baseline system's designs feature simplified connections with limited consideration for robustness and load-bearing capacity. In summary, Req2CAD could help designers enhance the overall quality of early-stage conceptual CAD.

RQ2: CAD Creation Support and User Experience Evaluation. The results in Table 6 indicate that, participants rated Req2CAD higher than the baseline in supporting broad design exploration (*Exploration*), expressing design intent (*Intent Expression*), aligning outputs with requirements (*Match*), and maintaining a clearer, more traceable workflow (*Transparency*). Participants were also more satisfied with the designs produced using Req2CAD (*Satisfaction*), echoing expert evaluations. In addition, Req2CAD was considered easier to use (*Ease of Use*) and more practically valuable for real-world design tasks (*Usefulness*). Statistical analysis revealed significant differences for all measures except *Match* and *Ease of use*. As shown in Figure 12a, the distribution of participant ratings further explains this: evaluations of Req2CAD were highly consistent, with predominantly positive scores (ranging from 3 to 5), whereas evaluations of the baseline system varied widely (ranging from 1 to 5). This discrepancy may stem from individual differences in participants' disciplinary expertise and proficiency with GPT and CAD systems (see Table 5). This indicates that Req2CAD provide

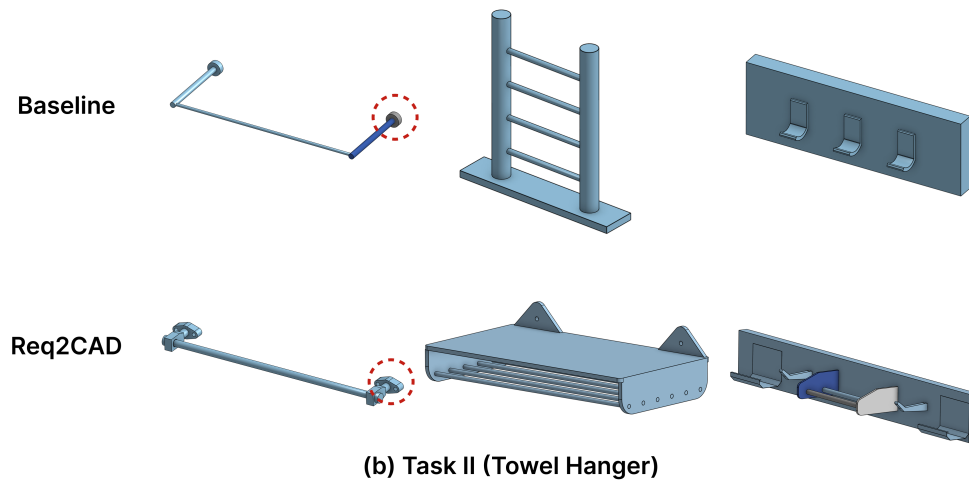
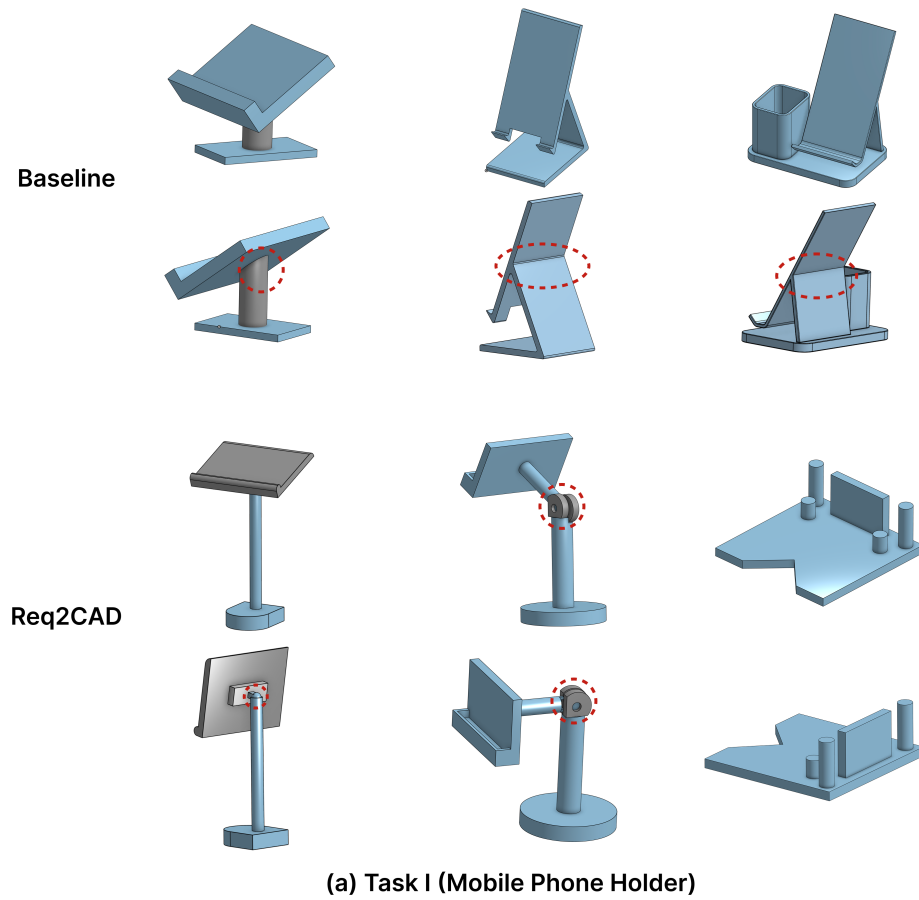


Figure 11: Example CAD models created by study participants using baseline system and Req2CAD.

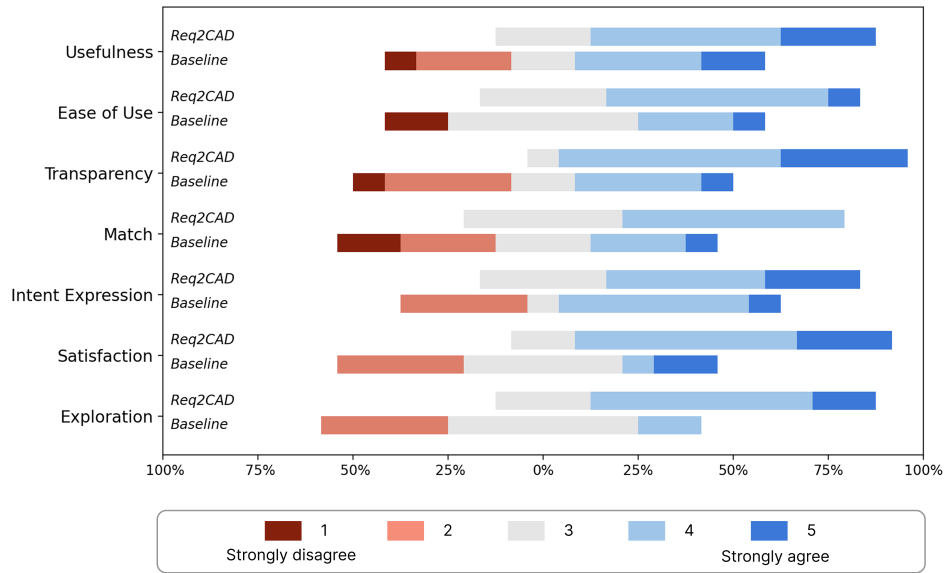
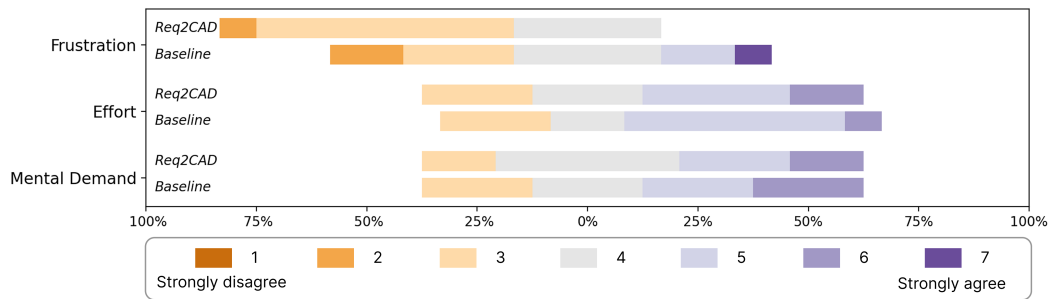
more stable support across with different backgrounds, highlighting its generalizability and adaptability. Table 7 and Figure 12b present the average scores and distributions of participants' task

load ratings for the two systems. The results show no significant differences in mental demand or effort, while participants reported higher frustration with the baseline. Conceptual CAD inherently

Table 6: The results showing average scores, standard deviations, and p-values of 5-point Likert scale responses on conceptual CAD support and human-AI co-creation experience across all participants for both Req2CAD and baseline system.

	Exploration	Satisfaction	Intent Expression	Match	Transparency	Ease of Use	Usefulness
Baseline	2.83 $\pm$ 0.69	3.08 $\pm$ 1.04	3.33 $\pm$ 1.03	2.83 $\pm$ 1.21	3.0 $\pm$ 1.15	3.08 $\pm$ 1.11	3.25 $\pm$ 1.23
Req2CAD	3.92 $\pm$ 0.64	4.08 $\pm$ 0.64	3.92 $\pm$ 0.76	3.58 $\pm$ 0.49	4.25 $\pm$ 0.60	3.75 $\pm$ 0.60	4.0 $\pm$ 0.71
Test Used	Wilcoxon	Paired t-test	Wilcoxon	Paired t-test	Wilcoxon	Paired t-test	Wilcoxon
p-value	0.016*	0.007**	0.034*	0.108	0.018*	0.087	0.014*

Note: ** denotes $p < 0.01$ and * denotes $p < 0.05$.

(a) Metrics for Conceptual CAD design Support and Human-AI Co-creation Experience**(b) Metrics for Task Load from NASA-TLX****Figure 12: Distribution of Likert scale responses on conceptual CAD support, human-AI co-creation experience, and task load across all participants for both Req2CAD and baseline system.**

imposes high cognitive and operational load, requiring designers not only to devise solutions that meet functional requirements but also to consider their parametric implementation. The capabilities of Req2CAD, such as function–structure reasoning and progressive

CAD generation with multi-modal input, help designers articulate their intent more effectively and obtain results that better align with their requirements, thereby reducing frustration.

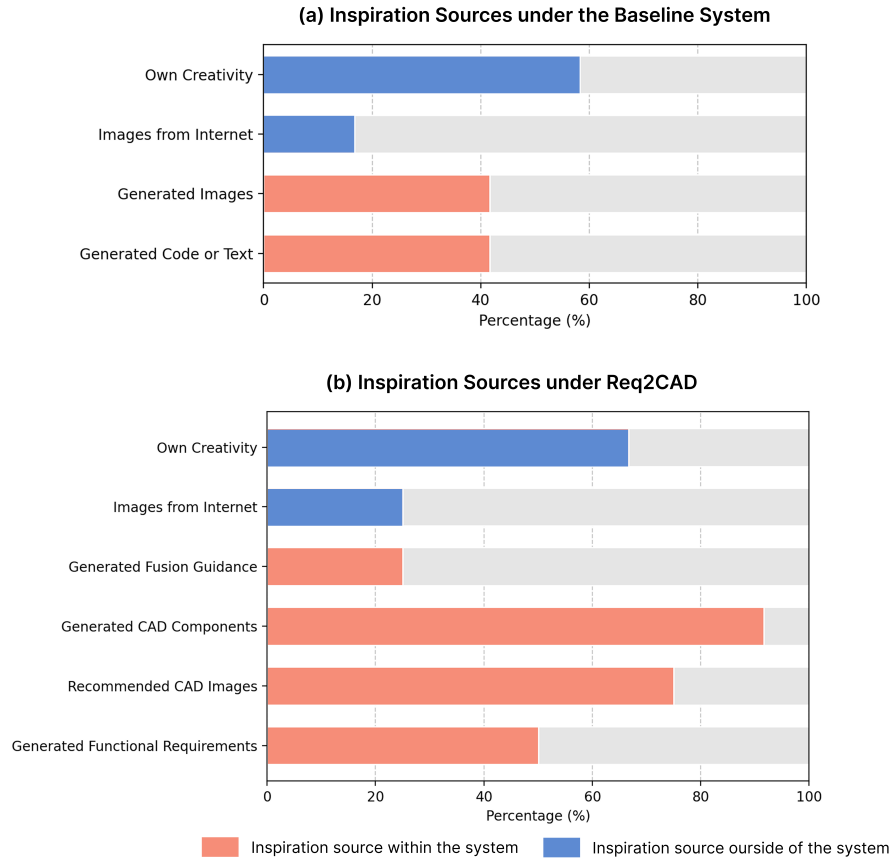


Figure 13: Proportion of participants inspired by different sources across Req2CAD and baseline system.

Table 7: The results showing average scores, standard deviations, and p-values of 7-point Likert scale responses on task load across all participants for both Req2CAD and baseline system.

	Mental Demand	Effort	Frustration
Baseline	4.5 ± 1.12	4.42 ± 0.95	3.83 ± 1.34
Req2CAD	4.42 ± 0.95	4.42 ± 1.04	3.25 ± 0.59
Test Used	Paired t-test	Paired t-test	Paired t-test
p-value	0.851	0.397	0.189

RQ3: Influence on concept ideation and design process.

Figure 13 presents the percentage of participants who reported that different kinds of outputs from the two systems effectively stimulated their creativity during the experiment. In the Baseline system, designers mainly relied on their own creativity, with GPT-generated outputs (e.g., CAD code, textual design descriptions, and product images) offering limited inspiration. In contrast, under the Req2CAD system, alongside “own creativity”, a wide range of generative outputs and recommended design cases were frequently reported as sources of inspiration. These results indicate that Req2CAD’s

progressive CAD generation and function–structure reasoning substantially broaden designers’ ideation space and the system offers more practically valuable sources of inspiration than the Baseline system.

Furthermore, based on our observation of the participants’ design process under the two systems, we found that, with the Baseline system, participants tended to request complete textual solutions or product images from ChatGPT, then either modeled a selected option in Onshape or iterated through prompts when dissatisfied. If repeated iterations still failed, some designers adopted a suboptimal output as a reference for manual CAD modeling, whereas others abandoned the AI-generated results entirely and relied on their own creativity. This approach limited independent reasoning about functionality and structural feasibility and constrained exploration to the few alternatives generated by ChatGPT. By contrast, Req2CAD’s function–structure reasoning encouraged participants to start from functional requirements and browse diverse design cases, supporting broader exploration and deeper reflection on structural feasibility. Designers then either edited retrieved CAD components to meet their needs or used them as inspiration to generate new CAD components with Req2CAD, reducing manual modeling efforts and improving efficiency. Additional examples

illustrating how participants used the two systems in their design process are provided in the supplementary materials.

6.3 Participants Feedback

6.3.1 Design Space Exploration and P-Creativity Stimulation. Most participants agreed that Req2CAD was more effective than the baseline system in expanding the design space and stimulating psychological creativity (P-creativity). It should be noted that P-creativity, or a P-creative idea, refers to an idea that is new to the person who generated it [8, 101]. First, the function-decomposition mechanism in Req2CAD revealed design aspects that participants had previously overlooked. For example, in the towel hanger task (Task II), P2 noted that his initial concept did not include a nut structure between the towel bar and the wall mount, but after seeing Req2CAD's decomposition of "provide structure support", he realized the need for fasteners to ensure stability, as shown in Column 1, Figure 11b. Second, many participants highlighted that ChatGPT typically produced only a few complete design schemes for direct selection, which often constrained their thinking—"just reproducing the given scheme in CAD"—and made it difficult to explore alternatives. In contrast, Req2CAD provided diverse CAD components aligned with functional requirements, enabling participants to explore design possibilities at the component level and combine alternatives more flexibly, which help break away from their personal habitual design patterns. For instance, P6 initially envisioned a single-hook towel rack, but through Req2CAD's recommendations discovered a composite structure integrating flat hanging rod with multiple hooks (see Column 3, Figure 11b). This function-structure reasoning encouraged deeper independent exploration and reflection, rather than confining participants to pre-packaged solutions. In addition, Req2CAD's visualization and clustering of recommended components using dual-feature CAD representations offered reusable structural patterns that accelerated ideation. As P9 observed, *"Req2CAD clusters together structures with similar structure, like perforated plates. These clusters highlight common patterns that may not be directly reusable but still offer useful references. Since the system recommends basic structural components tied to core functions, it doesn't restrict my later creative exploration. Instead, it gives me a solid starting point to build on."* By contrast, ChatGPT mainly delivered text or image outputs for specific tasks, lacking structured organization and offering limited support for divergent thinking.

6.3.2 Alignment Between Design Intent and Generated Results. Participants consistently noted that Req2CAD was more effective than the baseline system in helping them articulate their design intent and obtain matching results. ChatGPT showed clear limitations in understanding complex design requirements. Several participants emphasized that to get useful outputs, they had to provide overly long and precise text descriptions, which did not align with how designers typically communicate through diagrams, gestures, or quick sketches. As P1 remarked: *"When we communicate with other designers, we rely on lots of drawings and simple structural CAD models, but that's impossible with ChatGPT."* Others reported that when they attempted to refine and clarify ChatGPT's outputs by asking for multi-view 3D renderings or annotated 2D sketches,

the system repeatedly failed, often forcing them either to compromise with imperfect solutions (P6, P7, P9) or to abandon the tool and build new designs entirely on their own (P4). By contrast, Req2CAD significantly lowered the barrier to expressing intent through its function-structure reasoning and component recommendation mechanisms. With its rich part library and intuitive visualization features, designers did not need to rely solely on complex textual descriptions. Instead, they could browse, filter, iterate, and combine functional components to gradually refine and adjust their intent (P1, P3, P6). As P1 put it, even if their initial expression wasn't exact, *"I could still find structures that matched my needs pretty well."*

In addition, the multi-modal instruction for CAD generation and design flowchart provided by Req2CAD further enhanced the alignment between intent and results. P7 explained that graphical surface selection and iterative CAD generation *"bridges the gap between the AI and the designer, helping us point out spatial positions and iteration ideas more clearly, and aligning the AI's understanding with our intent."* P5 added that the design flowchart not only reduced the burden of typing instructions but also played a key role in managing the design process and tracking intent, making the workflow more transparent and controllable.

6.3.3 Ideation and CAD Modeling Efficiency. Participants reported that Req2CAD substantially improved design ideation and CAD modeling efficiency compared to the baseline. Its component recommendation and generation features reduced repetitive drawing work, enabling designers to focus on combining and refining ideas rather than manually reconstructing models. As P1 noted, when using ChatGPT, designers often had to manually reconstruct models based on text or image outputs, whereas Req2CAD *"already built most parts, and I only needed minor edits."* Similarly, P3 described Req2CAD's functional decomposition and function-structure reasoning as providing *"quick-start templates"* that supported fast iteration even without clear initial ideas. Some participants also tried using ChatGPT to generate text-based CAD modeling instructions, but found them limited. As P11 noted, while GPT offered structured guidance, its lack of spatial intuitiveness increased cognitive load, especially for novices. By contrast, Req2CAD directly generated 3D CAD models from prompts, allowing designers to *"focus on testing and refining ideas"* rather than wasting effort on manual modeling.

Moreover, Req2CAD's progressive CAD generation method and multi-model interaction were seen as greatly enhancing iteration efficiency and flexibility. As P9 explained, by selecting a surface and incrementally expanding structures, she could iterate very quickly, *"this progressive generation is more consistent with conventional CAD workflows."* P11 further added that this interaction style gave designers greater control over editing regions and modification directions. Moreover, the system supported deleting erroneous Python code segments and rolling back to previous versions, combining visual and code-based flexibility to make the overall design process both efficient and controllable.

6.3.4 Task Load and Willingness for Practical Use. According to participant feedback, the two systems imposed different types of mental demands on designers. The baseline system primarily required designers to expend mental effort figuring out how to model

complete design solutions generated by ChatGPT. In contrast, in Req2CAD, the mental demands arose from thinking about how to adapt the generated or recommended CAD components and how to assemble them. Participants generally agreed that, although the latter still demands mental effort, it offered greater autonomy and flexibility while ensuring higher-quality final solutions.

Regarding practical usage, most participants expressed willingness to use Req2CAD in their daily work. They noted that Req2CAD is particularly helpful in breaking fixed design patterns, expanding design approaches, and accelerating conceptual CAD modeling efficiency in practical design work. P2 further stated that ChatGPT is suitable for simple designs, but for complex design tasks, Req2CAD better supports engineering-oriented thinking and enables the rapid creation of solutions that meet contemporary practical standards. Participants also suggested improvements to Req2CAD: speeding up system response and improving integration with other CAD platforms such as SolidWorks and AutoCAD (P1, P11); optimizing the recommendation knowledge base to prioritize more useful structures and include more complex designs (P5, P10, P11); extending the design anchor-point guidance to automatic fusing and achieving automatic component assembly to reduce manual integration (P6, P11); and providing suggestions for structural improvements to aid design decisions (P2). These issues will be discussed in detail in Section 7.

7 Discussion

7.1 Implications on human-AI co-creation

While directly generating complete solutions provides quick visual outputs, it risks diminishing designers' independent thinking, particularly for novice designers with limited experience. In our study, a more experienced designer (p4) tended to reject ChatGPT's infeasible outputs and opted to design and create CAD model entirely on his own. In contrast, some less experienced participants (P1, P2, p5, p9) often chose to replicate ChatGPT-generated product images without carefully considering structural feasibility or comprehensive functional realization. This "passive acceptance" behavior risks fostering cognitive inertia and constraining designers' active exploration of the design space. By contrast, our proposed function-structure reasoning method granted designers greater agency during the design process. By recommending basic structural components rather than finished solutions, Req2CAD required participants to filter, combine, and refine these elements, and further evaluate whether the functional requirements were met. This process not only expanded the design space but also prompted deeper reasoning about constraints and feasibility.

In the long term, this mode may have two key implications. First, it may cultivate more structured design thinking: by selecting and assembling at the component level, novice designers naturally form a more nuanced understanding of the relationship between functions and structures, allowing them to treat AI recommendations as inspiration rather than final answers, thus improving overall design literacy. Second, as reflected in interviews, component-level reasoning promotes outcomes that participants perceived as more exploratory and personally innovative, since iterating over different combinations increases the likelihood of serendipitous discoveries. Furthermore, by starting from functional requirements and

grounding reasoning in structural components, this approach sustains designer agency while leveraging AI's supportive role. Such a balance fosters a healthier cycle of human-AI collaboration and mitigates the risk of long-term overreliance on AI that could otherwise erode creative capabilities [84].

7.2 Multi-modal Interaction for Intent Expression and Alignment

Our findings show that multi-modal interaction substantially improves human-AI intent alignment. Req2CAD enables designers to express intent through textual problem definitions, functional decomposition, and reference selections from 2D images of CAD cases, which is an approach similar to how human designers combine verbal explanations with sketches or 3D models in collaborative communication. While prior work has applied similar approaches to intent expression in 2D illustration [17] and product appearance design [14], our study demonstrates its effectiveness for conveying ambiguous intentions in more complex 3D conceptual design. For the unique challenges of 3D design, such as topological structures and spatial relationships, our results indicate that progressive CAD generation, which decomposes the realization of complex structures into multiple simpler steps, not only lowers the difficulty of intent expression for designers but also enhances GenAI's understanding. Designers also reported that this progressive generation process mirrors traditional manual modeling workflows, suggesting its potential for integration into real-world CAD modeling practices. In each step of iterative generation, the combination of textual descriptions with graphical interaction-based guidance was found to help designers articulate their iterative intentions more clearly and precisely. This aligns with previous empirical findings that multi-modal inputs support intent expression in 3D design iteration [30, 75].

7.3 Limitation and Future Work

Req2CAD enables broad applicability to product design tasks by decomposing design problems into basic functions and mapping them to fundamental CAD components. However, its scope is constrained by the coverage of the function-structure knowledge base it relies on. Although our current knowledge base already includes over 128k components, participants in the user study noted issues such as overly simple or repetitive structures. In future work, we plan to further filter and clean redundant structures, expand the knowledge base with more complex CAD components and assemblies drawn from real design projects, and apply our proposed annotation method for richer design knowledge labeling, thereby extending Req2CAD's applicability. Second, the computational creativity field distinguishes between two forms of novelty: psychological (P creativity) and historical (H creativity), with the latter representing ideas unprecedented in history [8, 101]. While participant feedback indicated that Req2CAD effectively supported novices in producing P-creative outcomes, its stimuli for structure ideation are mechanical CAD components extracted from existing design projects. Consequently, as reflected in the resulting outputs (as shown in Figure 11), Req2CAD rarely facilitated the creation of H-creative designs. A potential direction for fostering H-creativity is to incorporate inspiration sources beyond product and engineering

design domains—such as AskNature [3], a widely used biomimicry knowledge base that curates biological strategies, forms, and functional principles—to help designers explore more historically novel structural concepts. Third, Req2CAD currently focuses on recommending and generating component-level CAD models, leaving multi-component fusion and assembly to designers. Future research will explore how to support designers during the assembly stage by providing creative suggestions and concrete modeling strategies, for example, generating assembly solutions based on spatial and kinematic relations envisioned by the designer. Finally, we aim to further broaden the modes of design intent expression in CAD generation, such as gesture- or voice-based input, and investigate strategies for aligning and integrating these modalities, thereby providing more comprehensive support for intent expression and enhance the efficiency of CAD model creation and iteration.

8 Conclusion

We present Req2CAD, an interactive system for early-stage conceptual CAD model creation. Leveraging a function–structure knowledge base of over 128,000 annotated CAD components, it supports function decomposition, component-level structure derivation, and progressive CAD modeling. With our dual-feature CAD representation, the system allows efficient exploration of design spaces from geometric and topological perspectives and helps identify inspiring structural patterns. By combining multi-modal intent expression with an LLM-based progressive generation method, Req2CAD improves the design intent alignment between designer and LLM, and enhances the efficiency of CAD model generation and iteration. Technical evaluations validate the effectiveness of our GenAI-based function annotation pipeline and progressive CAD generation method. A user study with twelve novice designers further demonstrates the effectiveness of our system in improving both the quality of design outcomes and the human–AI co-creation experience. Finally, we discuss insights gained from the study and the limitations of Req2CAD, with the hope that our findings will inspire future research on intelligent support tools for 3D product design.

Acknowledgments

We thank all the participants for their time and the anonymous reviewers for their valuable comments. This research was supported by Information Technology Center and State Key Lab of CAD&CG, ZheJiang University.

References

- [1] Jorge Alcaide-Marzal, Jose Antonio Diego-Mas, and Gonzalo Acosta-Zazueta. 2020. A 3D shape generative method for aesthetic product design. *Design Studies* 66 (2020), 144–176. doi:10.1016/j.destud.2019.11.003
- [2] Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. 2025. Generating CAD Code with Vision-Language Models for 3D Designs. doi:10.48550/arXiv.2410.05340 arXiv:2410.05340 [cs].
- [3] AskNature. [n. d.]. *Nature Maintains Biodiversity — AskNature*. <https://asknature.org/strategy/nature-maintains-biodiversity/>
- [4] Autodesk. 2020. *Fusion Simulation Extension*. Retrieved August 31, 2025 from <https://www.autodesk.com/products/fusion-360/simulation-extension> Provides advanced simulation capabilities including structural, thermal, and generative design tools for Fusion 360 users..
- [5] Akshay Badagabettu, Sai Sravan Yarlagadda, and Amir Barati Farimani. 2024. Query2CAD: Generating CAD models using natural language queries. doi:10.48550/arXiv.2406.00144 arXiv:2406.00144 [cs].
- [6] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibozong, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zheng, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. arXiv:2502.13923 [cs.CV] <https://arxiv.org/abs/2502.13923>
- [7] M. P. Bendsoe and O. Sigmund. 1999. Material interpolation schemes in topology optimization. *Archive of Applied Mechanics* 69, 9 (01 Nov 1999), 635–654. doi:10.1007/s004190050248
- [8] Margaret A. Boden. 2009. Computer Models of Creativity. *AI Magazine* 30, 3 (Jul. 2009), 23. doi:10.1609/aimag.v30i3.2254
- [9] Stephen Brade, Bryan Wang, Mauricio Sousa, Sageev Oore, and Tovi Grossman. 2023. Promptify: Text-to-Image Generation through Interactive Prompt Exploration with Large Language Models. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) (UIST '23). Association for Computing Machinery, New York, NY, USA, Article 96, 14 pages. doi:10.1145/3586183.3606725
- [10] Zebin Cai, Rucong Liu, Qianzhi Jing, Haoyu Zuo, Lingyun Sun, Peter Childs, and Liqing Chen. 2025. An Automated Functional Decomposition Method Based on Large Language Models. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Vol. 89220. American Society of Mechanical Engineers, V03AT03A008.
- [11] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiang Xiao, Li Yi, and Fisher Yu. 2015. ShapeNet: An Information-Rich 3D Model Repository. arXiv:1512.03012 [cs.GR] <https://arxiv.org/abs/1512.03012>
- [12] Cheng Chen, Jiacheng Wei, Tianrun Chen, Chi Zhang, Xiaofeng Yang, Shangzhan Zhang, Bingchen Yang, Chuan-Sheng Foo, Guosheng Lin, Qixing Huang, and Fayao Liu. 2025. CADrafter: Generating Computer-Aided Design Models from Unconstrained Images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 11073–11082.
- [13] Liqing Chen, Wengteng Cheang, Zhaojun Jiang, Yuan Xu, Zebin Cai, Lingyun Sun, Peter Childs, Ji Han, Preben Hansen, and Haoyu Zuo. 2025. I-Card: A Generative AI-Supported Intelligent Design Method Card Deck. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (CHI '25). Association for Computing Machinery, New York, NY, USA, Article 426, 22 pages. doi:10.1145/3706598.3713934
- [14] Liqing Chen, Qianzhi Jing, Yixin Tsang, Qianyi Wang, Rucong Liu, Duowei Xia, Yunzhan Zhou, and Lingyun Sun. 2024. AutoSpark: Supporting Automobile Appearance Design Ideation with Kansei Engineering and Generative AI. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) (UIST '24). Association for Computing Machinery, New York, NY, USA, Article 108, 19 pages. doi:10.1145/3654777.3676337
- [15] Liqing Chen, Qianzhi Jing, Yixin Tsang, Qianyi Wang, Lingyun Sun, and Jianxi Luo. 2024. DesignFusion: Integrating Generative Models for Conceptual Design Enrichment. *Journal of Mechanical Design* 146, 11 (05 2024), 111703. doi:10.1115/1.4065487 arXiv:https://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/146/11/111703/7342975/md_146_11_111703.pdf
- [16] Xiang 'Anthony' Chen, Ye Tao, Guanyun Wang, Runchang Kang, Tovi Grossman, Stelian Coros, and Scott E. Hudson. 2018. Forte: User-Driven Generative Design. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3173574.3174070
- [17] DaEun Choi, Sumin Hong, Jeongeon Park, John Joon Young Chung, and Juho Kim. 2024. CreativeConnect: Supporting Reference Recombination for Graphic Design Ideation with Generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '24). Association for Computing Machinery, New York, NY, USA, Article 1055, 25 pages. doi:10.1145/3613904.3642794
- [18] Jiin Choi, Seung Won Lee, and Kyung Hoon Hyun. 2025. GenPara: Enhancing the 3D Design Editing Process by Inferring Users' Regions of Interest with Text-Conditional Shape Parameters. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–21. doi:10.1145/3706598.3713502
- [19] John Joon Young Chung and Eytan Adar. 2023. PromptPaint: Steering Text-to-Image Generation Through Paint Medium-like Interactions. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) (UIST '23). Association for Computing Machinery, New York, NY, USA, Article 6, 17 pages. doi:10.1145/3586183.3606777
- [20] FreeCAD Community. 2025. *FreeCAD*. <https://www.freecad.org/>
- [21] Lingchen Dai, Kang Zhang, Xianjun Sam Zheng, Ralph R. Martin, Yina Li, and Jinhui Yu. 2022. Visual complexity of shapes: a hierarchical perceptual learning model. *The Visual Computer* 38, 2 (01 Feb 2022), 419–432. doi:10.1007/s00371-020-02023-z

- [22] Dassault Systèmes. 2021. *Solidworks / Enhance Your Design Workflow with Generative Design*. Retrieved August 31, 2025 from <https://www.solidworks.com/media/enhance-your-design-workflow-generative-design> Recorded webinar (22 minutes); powered by SIMULIA's Tosca solvers.
- [23] Elona Dupont, Kseniya Cherenkova, Anis Kacem, Sk Aziz Ali, Ilya Arzhannikov, Gleb Gusev, and Djamila Aouada. 2022. CADOps-Net: Jointly Learning CAD Operation Types and Steps from Boundary-Representations. In *2022 International Conference on 3D Vision (3DV)*. 114–123. doi:10.1109/3DV57658.2022.00024
- [24] M. J. French. 1985. *Conceptual Design for Engineers*. Springer. doi:10.1007/978-3-662-11364-6
- [25] John S. Gero and Udo Kannengiesser. 2004. The situated function-behaviour-structure framework. *Design Studies* 25, 4 (2004), 373–391. doi:10.1016/j.destud.2003.10.010
- [26] Frederic Gmeiner, Humphrey Yang, Lining Yao, Kenneth Holstein, and Nikolas Martelaro. 2023. Exploring Challenges and Opportunities to Support Designers in Learning to Co-create with AI-based Manufacturing Design Tools. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 226, 20 pages. doi:10.1145/3544548.3580999
- [27] Haoxiang Guo, Shilin Liu, Hao Pan, Yang Liu, Xin Tong, and Baining Guo. 2022. ComplexGen: CAD reconstruction by B-rep chain complex generation. *ACM Trans. Graph.* 41, 4, Article 129 (July 2022), 18 pages. doi:10.1145/3528223.3530078
- [28] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*, Peter A. Hancock and Najmedin Meshkati (Eds.). Advances in Psychology, Vol. 52. North-Holland, 139–183. doi:10.1016/S0166-4115(08)62386-9
- [29] Julie Hirtz, Robert B. Stone, Daniel A. McAdams, Simon Szykman, and Kristin L. Wood. 2002. A functional basis for engineering design: Reconciling and evolving previous efforts. *Research in Engineering Design* 13, 2 (01 Mar 2002), 65–82. doi:10.1007/s00163-001-0008-3
- [30] Wen-Jun Hou, Ge-Xin Guo, and Yi-Ting Cheng. 2024. Multimodal Interaction Grammar Analysis Based on Two-Stage User-Based Elicitation in 3D Modeling. *International Journal of Human-Computer Interaction* 40, 8 (April 2024), 2120–2141. doi:10.1080/10447318.2023.2265735 Publisher: Taylor & Francis_eprint: <https://doi.org/10.1080/10447318.2023.2265735>.
- [31] R. Kenny Jones, Paul Guerrero, Niloy J. Mitra, and Daniel Ritchie. 2023. ShapeCoder: Discovering Abstractions for Visual Programs from Unstructured Primitives. *ACM Trans. Graph.* 42, 4, Article 49 (July 2023), 17 pages. doi:10.1145/3592416
- [32] Hyeonsu B Kang, David Chuan-En Lin, Yan-Ying Chen, Matthew K. Hong, Nikolas Martelaro, and Aniket Kittur. 2025. BioSpark: Beyond Analogical Inspiration to LLM-augmented Transfer. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (CHI '25). Association for Computing Machinery, New York, NY, USA, Article 653, 29 pages. doi:10.1145/3706598.3714053
- [33] Ahmet Serdar Karadeniz, Dimitrios Mallis, Nesryne Mejri, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. 2024. DAVINCI: A Single-Stage Architecture for Constrained CAD Sketch Inference. doi:10.48550/arXiv.2410.22857 arXiv:2410.22857 [cs].
- [34] Rubaiat Habib Kazi, Tov Grossman, Hyunmin Cheong, Ali Hashemi, and George Fitzmaurice. 2017. DreamSketch: Early Stage 3D Design Explorations with Sketching and Generative Design. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology* (Québec City, QC, Canada) (UIST '17). Association for Computing Machinery, New York, NY, USA, 401–414. doi:10.1145/3126594.3126662
- [35] Mohammad Sadi Khan, Elona Dupont, Sk Aziz Ali, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. 2024. CAD-SIGNet: CAD Language Inference from Point Clouds using Layer-wise Sketch Instance Guided Attention. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 4713–4722.
- [36] Mohammad S. Khan, Sankalp Sinha, Talha U. Sheikh, Didier Stricker, Sk A. Ali, and Muhammad Z. Afzal. 2024. Text2CAD: Generating Sequential CAD Designs from Beginner-to-Expert Level Text Prompts. *Advances in Neural Information Processing Systems* 37 (Dec. 2024), 7552–7579. https://proceedings.neurips.cc/paper_files/paper/2024/hash/0e5b96f97c1813bb75f6c28532c2ecc7-Abstract-Conference.html
- [37] Sumbul Khan and Bige Tunçer. 2019. Gesture and speech elicitation for 3D CAD modeling in conceptual design. *Automation in Construction* 106 (2019), 102847. doi:10.1016/j.autcon.2019.102847
- [38] Hui-Jun Kim, Jeongho Kim, Sohyun Jeong, Minbong Lee, Jaegul Choo, and Sung-Hee Kim. 2025. ShoeGenAI: A Creativity Support Tool for High-Feasible Shoe Product Design. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (CHI EA '25). Association for Computing Machinery, New York, NY, USA, Article 478, 11 pages. doi:10.1145/3706599.3721204
- [39] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. 2019. ABC: A Big CAD Model Dataset for Geometric Deep Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [40] Milin Kodnongbua, Benjamin Jones, Maaz Bin Safer Ahmad, Vladimir Kim, and Adriana Schulz. 2023. ReparamCAD: Zero-shot CAD Re-parameterization for Interactive Manipulation. In *SIGGRAPH Asia 2023 Conference Papers* (SA '23). Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3610548.3618219
- [41] Maksim Kolodiaznyi, Denis Tarasov, Dmitrii Zhemchuzhnikov, Alexander Nikulin, Ilya Zisman, Anna Vorontsova, Anton Konushin, Vladislav Kurenkov, and Danila Rukhovich. 2025. cadriple: Multi-modal CAD Reconstruction with Online Reinforcement Learning. doi:10.48550/arXiv.2505.22914 arXiv:2505.22914 [cs].
- [42] Juil Koo, Seungwoo Yoo, Minh Hieu Nguyen, and Minhyuk Sung. 2023. SALAD: Part-Level Latent Diffusion for 3D Shape Generation and Manipulation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 14441–14451.
- [43] Sivam Krish. 2011. A practical generative design method. *Computer-Aided Design* 43, 1 (2011), 88–100. doi:10.1016/j.cad.2010.09.009
- [44] Sangwon Lee and Jin Yan. 2016. The impact of 3D CAD interfaces on user ideation: A comparative analysis using SketchUp and Silhouette Modeler. *Design Studies* 44 (2016), 52–73. doi:10.1016/j.destud.2016.02.001
- [45] Songlin Li, Despoina Paschalidou, and Leonidas Guibas. 2024. PASTA: Controllable Part-Aware Shape Generation with Autoregressive Transformers. arXiv:2407.13677 [cs.CV] <https://arxiv.org/abs/2407.13677>
- [46] Xueyang Li, Yu Song, Yunzhong Lou, and Xiangdong Zhou. 2024. CAD Translator: An Effective Drive for Text to 3D Parametric Computer-Aided Design Generative Modeling. In *Proceedings of the 32nd ACM International Conference on Multimedia (MM '24)*. Association for Computing Machinery, New York, NY, USA, 8461–8470. doi:10.1145/3664647.3681549
- [47] Yuan Li, Cheng Lin, Yuan Liu, Xiaoxiao Long, Chenxu Zhang, Ningna Wang, Xin Li, Wenping Wang, and Xiaohu Guo. 2025. CADDreamer: CAD Object Generation from Single-view Images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 21448–21457.
- [48] David Chuan-En Lin, Hyeonsu B. Kang, Nikolas Martelaro, Aniket Kittur, Yan-Ying Chen, and Matthew K. Hong. 2025. Inkspire: Supporting Design Exploration with Generative AI through Analogical Sketching. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (CHI '25). Association for Computing Machinery, New York, NY, USA, Article 427, 18 pages. doi:10.1145/3706598.3713397
- [49] Vivian Liu, Jo Vermeulen, George Fitzmaurice, and Justin Matejka. 2023. 3DALL-E: Integrating Text-to-Image AI in 3D Design Workflows. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference* (Pittsburgh, PA, USA) (DIS '23). Association for Computing Machinery, New York, NY, USA, 1955–1977. doi:10.1145/3563657.3596098
- [50] Yujia Liu, Anton Obukhov, Jan Dirk Wegner, and Konrad Schindler. 2024. Point2CAD: Reverse Engineering CAD Models from 3D Point Clouds. 3763–3772. https://openaccess.thecvf.com/content/CVPR2024/html/Liu_Point2CAD_Reverse_Engineering_CAD_Models_from_3D_Point_Clouds_CVPR_2024_paper.html
- [51] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, Nov (2008), 2579–2605.
- [52] Dimitrios Mallis, Ahmet Serdar Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. 2025. CAD-Assistant: Tool-Augmented VLLMs as Generic CAD Task Solvers. arXiv:2412.13810 [cs.CV] <https://arxiv.org/abs/2412.13810>
- [53] Justin Matejka, Michael Glueck, Erin Bradner, Ali Hashemi, Tov Grossman, and George Fitzmaurice. 2018. Dream Lens: Exploration and Visualization of Large-Scale Generative Design Datasets. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3173574.3173943
- [54] Sangeun Oh, Yongsu Jung, Seongsin Kim, Ikjin Lee, and Namwoo Kang. 2019. Deep Generative Design: Integration of Topology Optimization and Generative Models. *Journal of Mechanical Design* 141, 11 (09 2019), 111405. doi:10.1115/1.4044229 arXiv:https://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111405/6578473/md_141_11_111405.pdf
- [55] OpenAI. 2024. GPT-4o System Card. arXiv:2410.21276 [cs.CL] <https://arxiv.org/abs/2410.21276>
- [56] OpenAI. 2025. DALL-E 3. Retrieved August 31, 2025 from <https://openai.com/dall-e-3>
- [57] Gerhard Pahl and Wolfgang Beitz. 2013. *Engineering design: a systematic approach*. Springer Science & Business Media.
- [58] Karran Pandey, Fanny Chevalier, and Karan Singh. 2023. Juxtaform: interactive visual summarization for exploratory shape design. *ACM Trans. Graph.* 42, 4 (July 2023), 52:1–52:14. doi:10.1145/3592436
- [59] Wamiq Para, Shariq Bhat, Paul Guerrero, Tom Kelly, Niloy Mitra, Leonidas J Guibas, and Peter Wonka. 2021. SketchGen: Generating Constrained CAD Sketches. In *Advances in Neural Information Processing Systems*, M. Ranzato,

- A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 5077–5088. https://proceedings.neurips.cc/paper_files/paper/2021/file/28891cb4ab421830acc36b1f5fd6c91e-Paper.pdf
- [60] Despoina Paschalidou, Ali Osman Ulusoy, and Andreas Geiger. 2019. Superquadrics Revisited: Learning 3D Shape Parsing Beyond Cuboids. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [61] PTC Inc. 2025. *Onshape*. Retrieved August 31, 2025 from <https://www.onshape.com/>
- [62] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. 2017. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [63] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>
- [64] B.F. Robertson and D.F. Radcliffe. 2009. Impact of CAD tools on creative problem solving in engineering design. *Computer-Aided Design* 41, 3 (March 2009), 136–146. doi:10.1016/j.cad.2008.06.007
- [65] Gianfranco La Rocca. 2012. Knowledge based engineering: Between AI and CAD. Review of a language based technology to support engineering design. *Advanced Engineering Informatics* 26, 2 (April 2012), 159–179. doi:10.1016/j.aei.2012.02.002
- [66] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis With Latent Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 10684–10695.
- [67] Robin Roy. 1993. Case studies of creativity in innovative product development. *Design Studies* 14, 4 (1993), 423–443. doi:10.1016/0142-694X(93)80016-6
- [68] Prabir Sarkar and Amaresh Chakrabarti. 2011. Assessing design creativity. *Design Studies* 32, 4 (2011), 348–383. doi:10.1016/j.destud.2011.01.002
- [69] Ari Seff, Yaniv Ovadia, Wenda Zhou, and Ryan P. Adams. 2020. SketchGraphs: A Large-Scale Dataset for Modeling Relational Geometry in Computer-Aided Design. arXiv:2007.08506 [cs.LG] <https://arxiv.org/abs/2007.08506>
- [70] Ari Seff, Wenda Zhou, Nick Richardson, and Ryan P. Adams. 2022. Vitruvion: A Generative Model of Parametric CAD Sketches. doi:10.48550/arXiv.2109.14124 arXiv:2109.14124 [cs].
- [71] Hanshu Shen, Lyukesheng Shen, Wenqi Wu, and Kejun Zhang. 2025. IdeationWeb: Tracking the Evolution of Design Ideas in Human-AI Co-Creation. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, 1–19. doi:10.1145/3706598.3713375
- [72] Gregory C. Smith and Shana Smith. 2012. Latent Semantic Engineering – A new conceptual user-centered design approach. *Advanced Engineering Informatics* 26, 2 (2012), 456–473. doi:10.1016/j.aei.2012.02.012
- [73] Kihoon Son, DaEun Choi, Tae Soo Kim, Young-Ho Kim, and Juho Kim. 2024. GenQuery: Supporting Expressive Visual Search with Generative Models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 180, 19 pages. doi:10.1145/3613904.3642847
- [74] Robert B Stone and Kristin L Wood. 1999. Development of a functional basis for design. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Vol. 19739. American Society of Mechanical Engineers, 261–275.
- [75] Lingyun Sun, Hongbo Zhang, Pei Chen, Zhaoqu Jiang, Xuelong Xie, Zihong Zhou, Xuanhui Liu, and Xiaoyu Chen. 2024. Elicitation and Evaluation of Hand-based Interaction Language for 3D Conceptual Design in Mixed Reality. *International Journal of Human-Computer Studies* 183 (2024), 103198. doi:10.1016/j.ijhcs.2023.103198
- [76] Dassault Systèmes. [n. d.]. *SolidWorks*. <https://www.solidworks.com/>
- [77] Jakob Tholander and Martin Jonsson. 2023. Design Ideation with AI - Sketching, Thinking and Talking with Generative Machine Learning Models. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference (Pittsburgh, PA, USA) (DIS '23)*. Association for Computing Machinery, New York, NY, USA, 1930–1940. doi:10.1145/3563657.3596014
- [78] Ryo Tsumoto, Kentaro Yaji, Yutaka Nomaguchi, and Kikuo Fujita. 2025. Deep concept identification for generative design. *Advanced Engineering Informatics* 65 (2025), 103354. doi:10.1016/j.aei.2025.103354
- [79] Michela Turrin, Peter von Buelow, and Rudi Stouffs. 2011. Design explorations of performance driven geometry in architectural design using parametric modeling and genetic algorithms. *Advanced Engineering Informatics* 25, 4 (2011), 656–675. doi:10.1016/j.aei.2011.07.009 Special Section: Advances and Challenges in Computing in Civil and Building Engineering.
- [80] Evangelos Tyflopoulos, Flem David Tollnes, Martin Steinert, Anna Olsen, et al. 2018. State of the art of generative design and topology optimization and potential research needs. *DS 91: Proceedings of NordDesign 2018, Linköping, Sweden, 14th–17th August 2018* (2018).
- [81] Karl T Ulrich, Steven D Eppinger, and Maria C Yang. 2008. *Product design and development*. Vol. 4. McGraw-Hill higher education Boston.
- [82] Cyrus Vachha, Yixiao Kang, Zach Dive, Ashwat Chidambaram, Anik Gupta, Eunice Jun, and Björn Hartmann. 2025. Dreamcrafter: Immersive Editing of 3D Radiance Fields Through Flexible, Generative Inputs and Outputs. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3706598.3714312
- [83] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [84] Samangi Wadinambiarachchi, Ryan M. Kelly, Saumya Pareek, Qiushi Zhou, and Eduardo Velloso. 2024. The Effects of Generative AI on Design Fixation and Divergent Thinking. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 380, 18 pages. doi:10.1145/3613904.3642919
- [85] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. 2019. Dynamic Graph CNN for Learning on Point Clouds. *ACM Trans. Graph.* 38, 5, Article 146 (Oct. 2019), 12 pages. doi:10.1145/3326362
- [86] Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. 2021. Fusion 360 gallery: a dataset and environment for programmatic CAD construction from human design sequences. *ACM Trans. Graph.* 40, 4, Article 54 (July 2021), 24 pages. doi:10.1145/3450626.3459818
- [87] Randall H. Wilson and Jean-Claude Latombe. 1994. Geometric reasoning about mechanical assembly. *Artificial Intelligence* 71, 2 (Dec. 1994), 371–396. doi:10.1016/0004-3702(94)90048-5
- [88] Rundi Wu, Chang Xiao, and Changxi Zheng. 2021. DeepCAD: A Deep Generative Network for Computer-Aided Design Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 6772–6782.
- [89] Xiang Xu, Pradeep Kumar Jayaraman, Joseph G. Lambourne, Karl D. D. Willis, and Yasutaka Furukawa. 2023. Hierarchical Neural Coding for Controllable CAD Model Generation. doi:10.48550/arXiv.2307.00149 arXiv:2307.00149 [cs].
- [90] Xianghao Xu, Wenzhe Peng, Chin-Yi Cheng, Karl D.D. Willis, and Daniel Ritchie. 2021. Inferring CAD Modeling Sequences Using Zone Graphs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 6062–6070.
- [91] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuxiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [92] Kaizhi Yang and Xuejin Chen. 2021. Unsupervised learning for cuboid shape abstraction via joint segmentation from point clouds. *ACM Trans. Graph.* 40, 4, Article 152 (July 2021), 11 pages. doi:10.1145/3450626.3459873
- [93] Xue Yu, Stephen DiVerdi, and Yotam Gingold. 2025. A Scaffold-Based Tool for Product Design Variations in Virtual Reality. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/3706598.3713816
- [94] Loutfouz Zaman, Christian Neugebauer, Wolfgang Stuerzlinger, and Robert Woodbury. 2018. GEM-NI+: Leveraging Difference Visualization and Multiple Displays for Supporting Multiple Complex Generative Design Alternatives. In *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems (Montreal QC, Canada) (CHI EA '18)*. Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3170427.3188593
- [95] Biao Zhang, Jiapeng Tang, Matthias Nießner, and Peter Wonka. 2023. 3DShape2VecSet: A 3D Shape Representation for Neural Fields and Generative Diffusion Models. *ACM Trans. Graph.* 42, 4, Article 92 (July 2023), 16 pages. doi:10.1145/3592442
- [96] Hongbo Zhang, Pei Chen, Xuelong Xie, Zhaoqu Jiang, Yifei Wu, Zejian Li, Xiaoyu Chen, and Lingyun Sun. 2025. FusionProtor: A Mixed-Prototype Tool for Component-level Physical-to-Virtual 3D Transition and Simulation. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, 1–19. doi:10.1145/3706598.3713686
- [97] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. 2023. Adding Conditional Control to Text-to-Image Diffusion Models. In *Proceedings of the IEEE/CVF*

- International Conference on Computer Vision (ICCV)*. 3836–3847.
- [98] Yongjie Zhang, Kang Cao, Zhiqiang Cai, Xinwen Wang, and Wei Jin. 2021. Intelligent Matching Assembly: System Design Based on Reuse of Ultra-Difference Parts. *International Journal of Precision Engineering and Manufacturing* 22, 7 (July 2021), 1205–1220. doi:10.1007/s12541-021-00531-z
- [99] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. arXiv:2506.05176 [cs.CL] <https://arxiv.org/abs/2506.05176>
- [100] Zhanwei Zhang, Shizhao Sun, Wenxiao Wang, Deng Cai, and Jiang Bian. 2025. FlexCAD: Unified and Versatile Controllable CAD Generation with Fine-tuned Large Language Models. arXiv:2411.05823 [cs.CV] <https://arxiv.org/abs/2411.05823>
- [101] Eric B. Zhou, Dokyun Lee, and Bin Gu. 2025. Who expands the human creative frontier with generative AI. *Science Advances* 11, 36 (2025), eadu5800. doi:10.1126/sciadv.adu5800
- [102] Qingnan Zhou and Alec Jacobson. 2016. Thingi10K: A Dataset of 10,000 3D-Printing Models. arXiv:1605.04797 [cs.GR] <https://arxiv.org/abs/1605.04797>
- [103] Shengdi Zhou, Tianyi Tang, and Bin Zhou. 2023. CADParser: A Learning Approach of Sequence Modeling for B-Rep CAD. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, Macau, SAR China, 1804–1812. doi:10.24963/ijcai.2023/200